

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ingeniería

Diplomado en Infraestructura en Tecnologías de la Información
(División de Educación Continua – FI UNAM)

Proyecto Final

Integración de Identidad: Keycloak + OIDC + Kubernetes RBAC

Módulo: Seguridad en Informática

Integrantes del equipo:

- Saúl Peralta Pérez
 - Rodrigo Beristain Martinez
 - Jessica Arellano Marquez
-

Fecha: Marzo 2026

Sede: Facultad de Ingeniería, UNAM

Requerimientos del proyecto

Integración de Identidad: Keycloak + OIDC + Kubernetes RBAC

Módulo 1 — Despliegue de Keycloak en Kubernetes

Propósito

Implementar Keycloak dentro del clúster para utilizarlo como proveedor de identidad central.

Requerimientos

- Crear el namespace keycloak
- Crear un Secret con las credenciales administrativas
- Desplegar Keycloak en modo desarrollo con base de datos H2 embebida
- Exponer el servicio mediante **NodePort**

Resultado esperado

- Keycloak accesible desde navegador
- Consola administrativa funcional
- Pod en estado **Running**

Entregables

- Keycloak corriendo y accesible en `http://192.168.109.200:30097`
- Captura de pantalla de la consola de administración
- Salida de `kubectl describe pod -n keycloak -l app=keycloak`

Módulo 2 — Configuración de Keycloak

Propósito

Definir la estructura lógica de identidad que usará Kubernetes: realm, client, grupos y usuarios.

Requerimientos

- Crear el realm kubernetes-demo
- Crear el client [kubernetes](#)
- Configurar OIDC con autenticación tipo confidential
- Crear el mapper del claim [groups](#)
- Crear los grupos:
 - k8s-ops
 - k8s-developers
 - k8s-readonly
- Crear los usuarios:
 - ops-user
 - dev-user
 - viewer

Resultado esperado

- Los usuarios deben pertenecer a sus grupos respectivos
- El token JWT debe incluir el claim [groups](#)

Entregables

- Capturas de pantalla del Realm, Client, Grupos y Usuarios
- Token JWT decodificado en [jwt.io](#)
- Evidencia visual del claim [groups](#)

Módulo 3 — Configuración de kube-apiserver con OIDC

Propósito

Integrar Kubernetes con Keycloak para aceptar autenticación basada en tokens OIDC.

Requerimientos

- Editar el manifiesto estático de kube-apiserver
- Agregar flags OIDC:
 - `--oidc-issuer-url`
 - `--oidc-client-id`
 - `--oidc-username-claim`
 - `--oidc-groups-claim`
 - `--oidc-groups-prefix`
- Validar que el API Server reinicie correctamente
- Confirmar que el clúster sigue operativo

Resultado esperado

- El API Server debe iniciar sin errores OIDC
- Kubernetes debe seguir respondiendo a `kubectl`

Entregables

- Captura del kube-apiserver.yaml con los flags OIDC
- Salida de `kubectl cluster-info`
- Logs del API Server relacionados con OIDC

Módulo 4 — RBAC mapeado a grupos de Keycloak

Propósito

Aplicar autorización en Kubernetes con base en grupos administrados desde Keycloak.

Requerimientos

- Crear un ClusterRoleBinding para k8s-ops
- Crear un namespace **proyecto-dev**
- Crear un **RoleBinding** para **k8s-developers** sobre **proyecto-dev**
- Crear un **ClusterRoleBinding** para **k8s-readonly**

Resultado esperado

- **ops-user** debe tener acceso total al clúster
- **dev-user** debe tener permisos de edición únicamente en **proyecto-dev**
- **viewer** debe tener permisos de solo lectura

Entregables

- Manifiestos o evidencias de los 3 bindings
- Tabla de permisos:

Usuario	Grupo Keycloak	Permisos esperados
ops-user	k8s-ops	Acceso total al clúster (cluster-admin)
dev-user	k8s-developers	CRUD en namespace proyecto-dev
viewer	k8s-readonly	Solo lectura en todo el clúster

Módulo 5 — Autenticación con kubectl vía OIDC

Propósito

Permitir que los usuarios se autenticuen en Kubernetes usando OIDC, sin certificados manuales ni cuentas locales.

Requerimientos

- Instalar kubelogin
- Obtener el **Client Secret** del cliente **kubernetes**
- Configurar credenciales OIDC con **kubectl config set-credentials**
- Crear el contexto **dev-context**
- Validar acceso con **dev-user**

Resultado esperado

- Inicio de sesión exitoso en Keycloak desde kubectl
- Acceso permitido al namespace **proyecto-dev**
- Acceso denegado a recursos no autorizados

Entregables

- Salida de **kubectl config get-contexts**
- Evidencia del login OIDC
- Evidencia de autorización y denegación de acceso

Módulo 6 — Cambio dinámico de permisos desde Keycloak

Propósito

Demostrar que el cambio de grupo en Keycloak impacta inmediatamente en los permisos dentro de Kubernetes.

Requerimientos

- Verificar permisos limitados de dev-user
- Mover `dev-user` del grupo `k8s-developers` a `k8s-ops`
- Forzar obtención de un nuevo token
- Validar que el usuario adquiere permisos de administrador

Resultado esperado

- Antes del cambio: acceso restringido
- Después del cambio: acceso ampliado conforme al nuevo grupo

Entregables

- Evidencia del cambio de grupo en Keycloak
- Evidencia del nuevo comportamiento de permisos en Kubernetes

Módulo 7 — Observabilidad: Jaeger + Fluent Bit + Loki

Propósito

Incorporar observabilidad al proyecto mediante trazas distribuidas y centralización de logs.

Requerimientos 7a. Jaeger

- Crear namespace tracing
- Desplegar Jaeger
- Exponer Jaeger UI por **NodePort**

Resultado esperado

- UI de Jaeger accesible
- Evidencia visual del servicio corriendo

Entregables

- Jaeger UI corriendo en NodePort 30100
- Captura de la interfaz
- Validación de pods y servicios

Requerimientos 7b. Fluent Bit + Loki + Grafana

- Crear namespace logging
- Instalar Loki Stack con Helm
- Habilitar Grafana
- Habilitar Fluent Bit
- Desplegar el stack de logging en el clúster

Resultado esperado

- Loki y Grafana operativos
- Fluent Bit desplegado como recolector de logs
- Visualización de logs de Keycloak en Grafana o, en su defecto, validación técnica de Loki y del pipeline de logging

Entregables

- Evidencia de Loki y Grafana desplegados
- Evidencia de Fluent Bit funcionando
- Querys útiles propuestas:
 - `{namespace="keycloak"} |= "login"`
 - `{namespace="keycloak"} |= "error"`

Entregables finales del Equipo 3

Documentación

- Diagrama de flujo completo: usuario → Keycloak → JWT → Kubernetes RBAC
- Tabla de usuarios, grupos, roles y permisos
- Documento comparativo: OIDC/Keycloak vs certificados X.509 manuales
 - ventajas
 - desventajas
 - casos de uso

Técnico

- Keycloak corriendo con Realm, Client, 3 Grupos y 3 Usuarios
- `kube-apiserver` configurado con OIDC
- 3 RoleBindings / ClusterRoleBindings mapeados a grupos de Keycloak
- `kubectl` funcional con autenticación OIDC
- Demo de cambio de grupo en Keycloak y cambio de permisos en Kubernetes
- Jaeger corriendo con trazas de autenticación
- Fluent Bit + Loki desplegados para centralización de logs

Presentación

- **5 minutos:** Demo de login OIDC en Keycloak y uso de `kubectl`
 - **5 minutos:** Demo de 3 usuarios con diferentes niveles de acceso
 - **5 minutos:** Jaeger y Loki mostrando trazas y eventos de autenticación
-

Resumen técnico del stack de observabilidad

Tracing

- Jaeger
 - Recolecta trazas de autenticación
 - Permite visualizar flujos OIDC
 - Facilita el análisis de errores y latencias

Logging

- **Fluent Bit**
 - Recolecta logs del clúster
- **Loki**
 - Almacena logs
- **Grafana**
 - Visualiza logs de aplicaciones y componentes

Componentes y puertos

Componente	Namespace	Puerto / Acceso
Keycloak	keycloak	NodePort 30097
Jaeger UI	tracing	NodePort 30100
Grafana	logging	NodePort 30101
Loki	logging	NodePort 30102
Fluent Bit	logging	DaemonSet

Recursos de apoyo

- Keycloak
- Kubernetes OIDC Authentication
- kubelogin
- Jaeger Operator
- Fluent Bit + Loki
- JWT Decoder ([jwt.io](#))

• Objetivo general

Desplegar **Keycloak** como proveedor de identidad **OAuth2/OpenID Connect (OIDC)** dentro del clúster de Kubernetes, integrarlo con el **Kubernetes API Server** para autenticación centralizada de usuarios, y mapear grupos de Keycloak a roles **RBAC** de Kubernetes, demostrando un modelo de identidad empresarial moderno, centralizado y escalable.

Arquitectura objetivo

La arquitectura propuesta busca centralizar la autenticación de usuarios mediante Keycloak, emitir tokens JWT vía OIDC y permitir que el Kubernetes API Server valide dichos tokens para aplicar permisos según los grupos asignados en Keycloak.

Flujo general

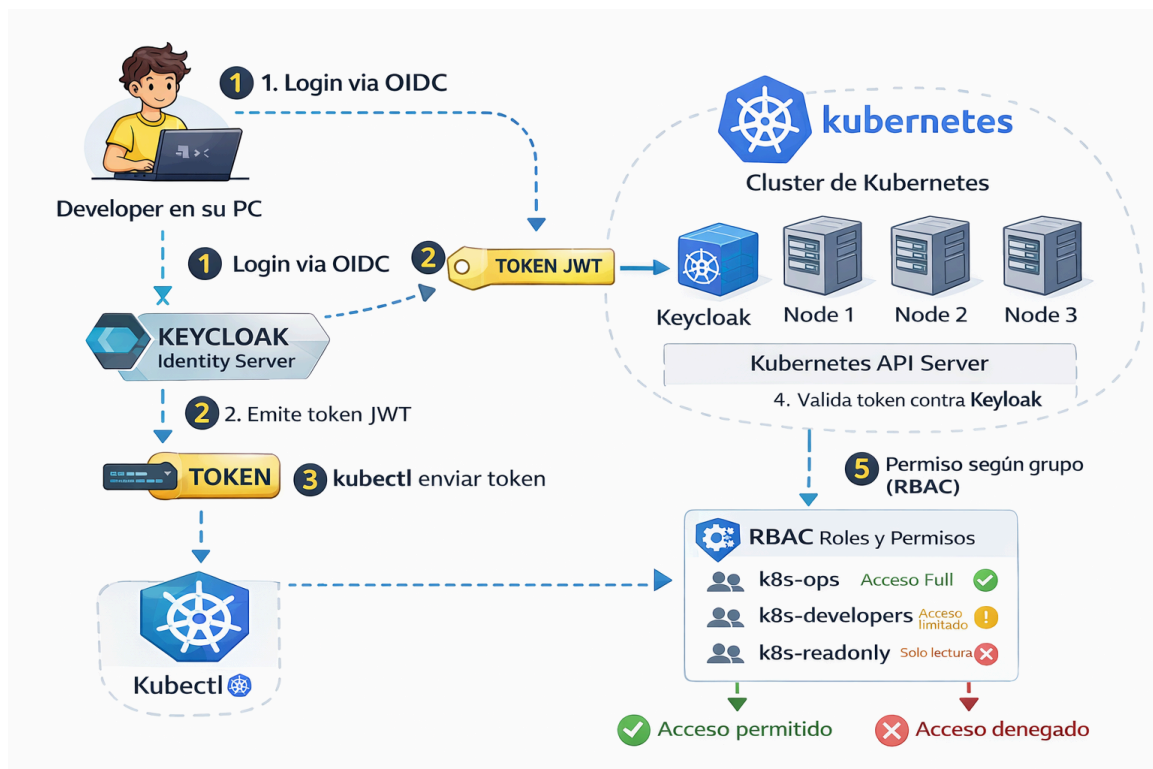
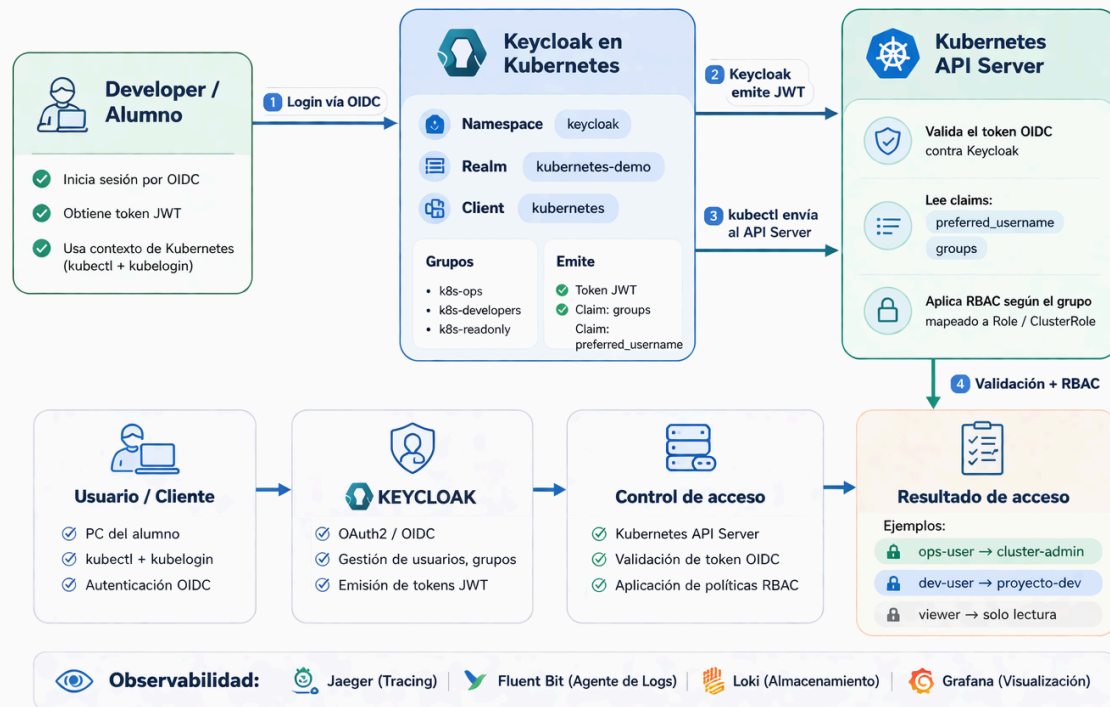
1. El usuario inicia sesión desde su equipo usando `kubectl` con `kubelogin`.
2. Keycloak autentica al usuario y emite un token JWT.
3. `kubectl` envía ese token al Kubernetes API Server.
4. El API Server valida el token contra Keycloak.
5. Kubernetes aplica las políticas RBAC según el grupo recibido en el claim `groups`.
6. El acceso es permitido o denegado en función del rol asignado.

Componentes principales

- **Developer / Alumno**
 - Usa `kubectl` y `kubelogin`
 - Se autentica mediante OIDC
 - Consume el clúster según los permisos recibidos
- **Keycloak en Kubernetes**
 - Desplegado en el namespace `keycloak`
 - Expone autenticación OIDC
 - Administra usuarios, clientes, grupos y emisión de tokens
- **Kubernetes API Server**
 - Valida tokens OIDC
 - Lee claims del token como `preferred_username` y `groups`
 - Aplica autorización mediante RBAC
- **Observabilidad**
 - **Jaeger** para trazabilidad de flujos
 - **Fluent Bit + Loki** para recolección y almacenamiento de logs
 - **Grafana** para visualización

Arquitectura objetivo – Keycloak + OIDC + Kubernetes RBAC

Equipo 3 · Proyecto Final · Seguridad en Infraestructura y Kubernetes



Creación de cluster Kubernetes con 3 VMs conectadas.

Desactivar swap: se desactiva el swap porque Kubernetes necesita controlar exactamente cuánta memoria usan los pods. Si el sistema usa swap, ese control se rompe. (En las 3 mv)

```
swapoff -a
sed -i '/swap/d' /etc/fstab
```

Cargamos modprobe br_netfilter: Este módulo permite que el tráfico de red que pasa por bridges (puentes de red) pueda ser procesado por iptables o nftables.

```
modprobe br_netfilter
```

Crea un archivo de configuración del kernel: para ajustar parámetros de red que necesita Kubernetes.

```
cat <<EOF > /etc/sysctl.d/k8s.conf
    net.bridge.bridge-nf-call-iptables = 1

    net.ipv4.ip_forward = 1

    net.bridge.bridge-nf-call-ip6tables = 1

EOF
```

Aplica los cambios:

```
sysctl --system
```

Instalar container runtime: containerd desde el repositorio docker

```
dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
dnf makecache
```

```
dnf install -y containerd.io
```

Inicializa el servicio:

```
systemctl enable --now containerd
```

Instalar Kubernetes: Agregar repositorio

```
cat <<EOF > /etc/yum.repos.d/kubernetes.repo
[kubernetes]

    name=Kubernetes

baseurl=https://pkgs.k8s.io/core:/stable:/v1.29/rpm/
enabled=1
gpgcheck=1
gpgkey=https://pkgs.k8s.io/core:/stable:/v1.29/rpm/repodata/repomd.xml.key
EOF
```

Instalar los paquetes:

```
dnf install -y kubelet kubeadm kubectl
```

Inicializar kubelet:

```
systemctl enable --now kubelet
```

Inicializar el MASTER

```
kubeadm init --pod-network-cidr=192.168.0.0/16
```

Configurar kubectl

```
mkdir -p $HOME/.kube  
cp /etc/kubernetes/admin.conf $HOME/.kube/config
```

Fase 1: Despliegue de Keycloak (En la MV Master)

Crea el namespace **keycloak** :

```
kubect1 create namespace keycloak
```

Crear credenciales de administrador **#Creando un Secret de Kubernetes**

```
cd ~/proyecto-oidc
```

```
kubect1 create secret generic keycloak-admin \
--from-literal=KEYCLOAK_ADMIN=admin \
--from-literal=KEYCLOAK_ADMIN_PASSWORD=Admin1234! \
-n keycloak
```

Crear Deployment de Keycloak

```
cd ~/proyecto-oidc/keycloak #Aquí estamos desplegando Keycloak dentro de Kubernetes.
```

```
vi keycloak-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: keycloak
  namespace: keycloak
spec:
  replicas: 1
  selector:
    matchLabels:
      app: keycloak
  template:
    metadata:
      labels:
        app: keycloak
    spec:
      containers:
```

- name: keycloak

```
image: quay.io/keycloak/keycloak:23.0
args: ["start-dev"]
env:
```

- name: KEYCLOAK_ADMIN

```
valueFrom:
  secretKeyRef:
    name: keycloak-admin
    key: KEYCLOAK_ADMIN
```

- name: KEYCLOAK_ADMIN_PASSWORD

```
valueFrom:
```

```

      secretKeyRef:
        name: keycloak-admin
        key: KEYCLOAK_ADMIN_PASSWORD
    • name: KC_HTTP_PORT
      value: "8080"
      ports:
    • containerPort: 8080

    resources:
      requests:
        cpu: "250m"

        memory: "512Mi"

      limits:
        cpu: "1"

        memory: "1Gi"

```

vi keycloak-service.yaml # Crear Service NodePort, un Service expone un pod, puedes acceder desde fuera del cluster

```

      apiVersion: v1
      kind: Service
      metadata:
        name: keycloak
        namespace: keycloak
      spec:
        type: NodePort
        selector:
          app: keycloak

        ports:
    • port: 8080

      targetPort: 8080

      nodePort: 30097

```

Desplegar Keycloak #crea estos recursos

```

kubect1 apply -f keycloak-deployment.yaml
kubect1 apply -f keycloak-service.yaml

```

Verificar pod

```
kubect1 get pods -n keycloak -o wide
```

Verificar service

```
kubect1 get svc -n keycloak
```

Acceder a Keycloak

```

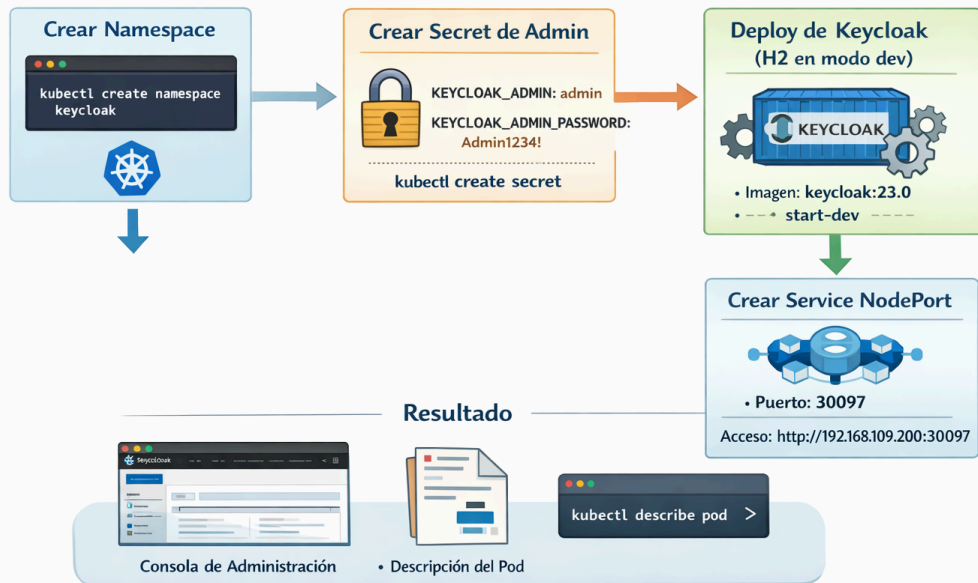
http://192.168.109.202:30097
  user: admin

  password: Admin1234!

```

Módulo 1

Desplegar Keycloak en Kubernetes



Fase 2: Crear Realm

Create Realm

Create realm

A realm manages a set of users, credentials, roles, and groups. A user belongs to and logs into a realm. Realms are isolated from one another and can only manage and authenticate the users that they control.

Resource file

Drag a file here or browse to upload Browse... Clear

1

Upload a JSON file

Realm name *

Crear cliente

Clients > Client details

kubernetes OpenID Connect Enabled Action

Clients are applications and services that can request authentication of a user.

Settings **Roles** **Client scopes** **Sessions** **Advanced**

General settings

Client ID *

Name

Description

Save Revert

Jump to section

- General settings
- Access settings
- Capability config
- Login settings

Crear grupos

Groups

A group is a set of attributes and role mappings that can be applied to a user. You can create, edit, and delete groups and manage their child-parent organization. [Learn more](#)

→

☐ Exact search

1 - 3

- k8s-developers
- k8s-ops
- k8s-readonly

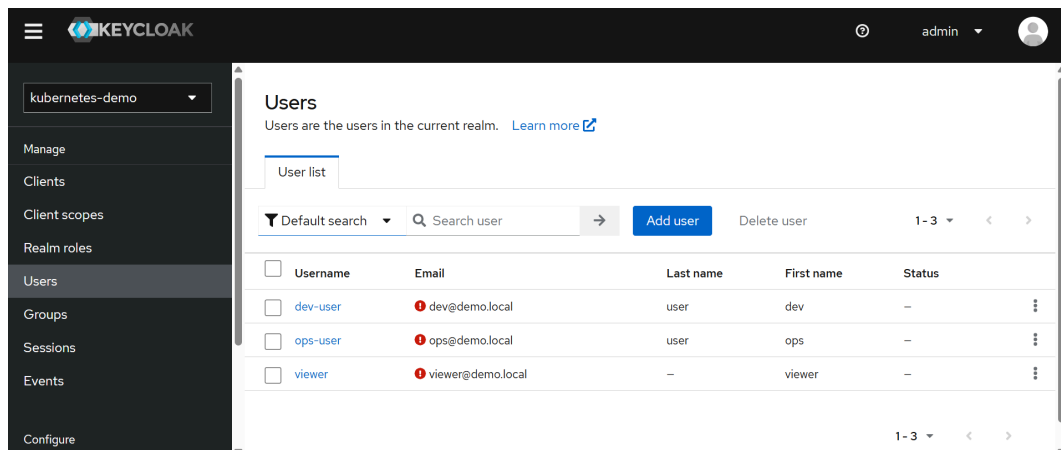
1 - 3

→ Create group

1 - 3

- ☐ Group name
- ☐ k8s-developers
- ☐ k8s-ops
- ☐ k8s-readonly

Crear usuarios



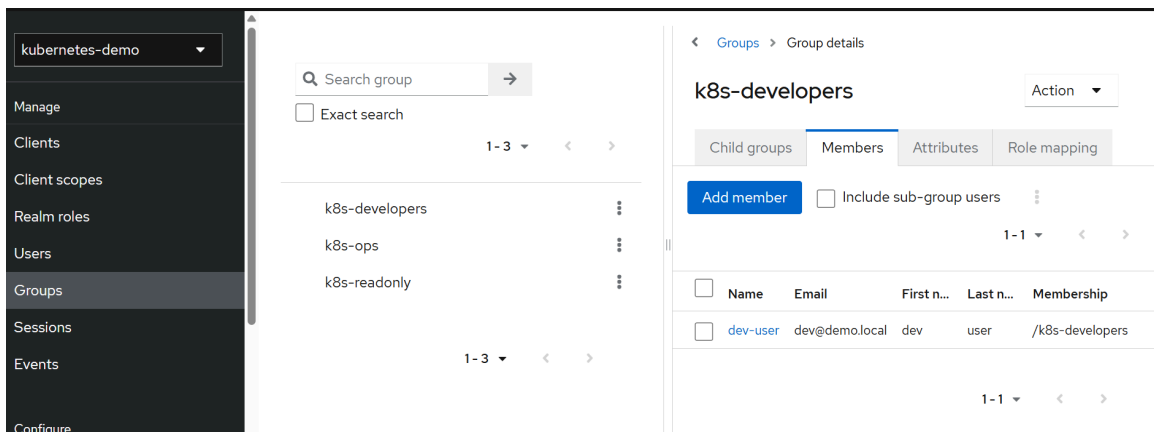
Users
Users are the users in the current realm. [Learn more](#)

User list

Default search Search user Add user Delete user 1-3

☐	Username	Email	Last name	First name	Status
☐	dev-user	dev@demo.local	user	dev	—
☐	ops-user	ops@demo.local	user	ops	—
☐	viewer	viewer@demo.local	—	viewer	—

1-3



Search group Search group Exact search 1-3

k8s-developers
k8s-ops
k8s-readonly

1-3

< Groups > Group details

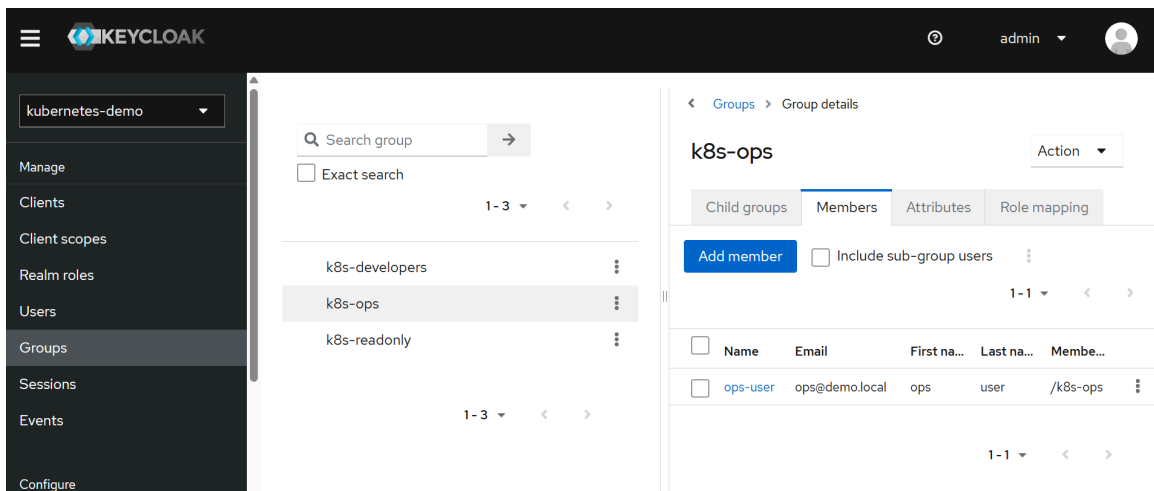
k8s-developers Action

Child groups Members Attributes Role mapping

Add member Include sub-group users 1-1

☐	Name	Email	First n...	Last n...	Membership
☐	dev-user	dev@demo.local	dev	user	/k8s-developers

1-1



Search group Search group Exact search 1-3

k8s-developers
k8s-ops
k8s-readonly

1-3

< Groups > Group details

k8s-ops Action

Child groups Members Attributes Role mapping

Add member Include sub-group users 1-1

☐	Name	Email	First na...	Last na...	Membe...
☐	ops-user	ops@demo.local	ops	user	/k8s-ops

1-1

☰

KEYCLOAK

kubernetes-demo

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

🔍 Search group

➔

☐ Exact search

1 - 3

<

>

k8s-developers

⋮

k8s-ops

⋮

k8s-readonly

⋮

1 - 3

<

>

< Groups > Group details

k8s-readonly

Action

Child groups

Members

Attributes

Role mapping

Add member

☐ Include sub-group users

1 - 1

<

>

☐

Name

☐

viewer

☐

viewer@demo.local

☐

viewer

☐

-

☐

/k8s-readonly

1 - 1

<

>

[illegible][illegible][illegible]

<https://www.jwt.io/>

Usuario ops-user

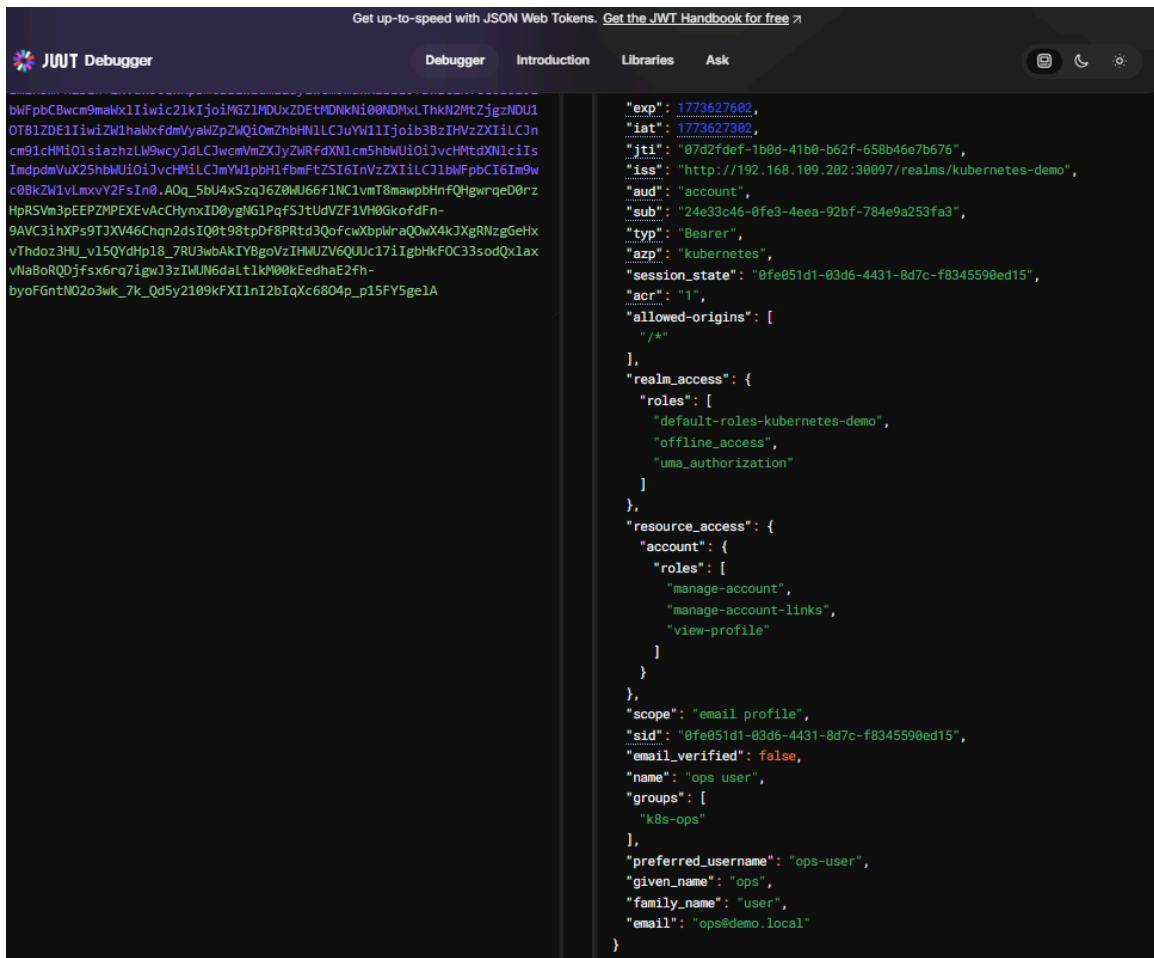
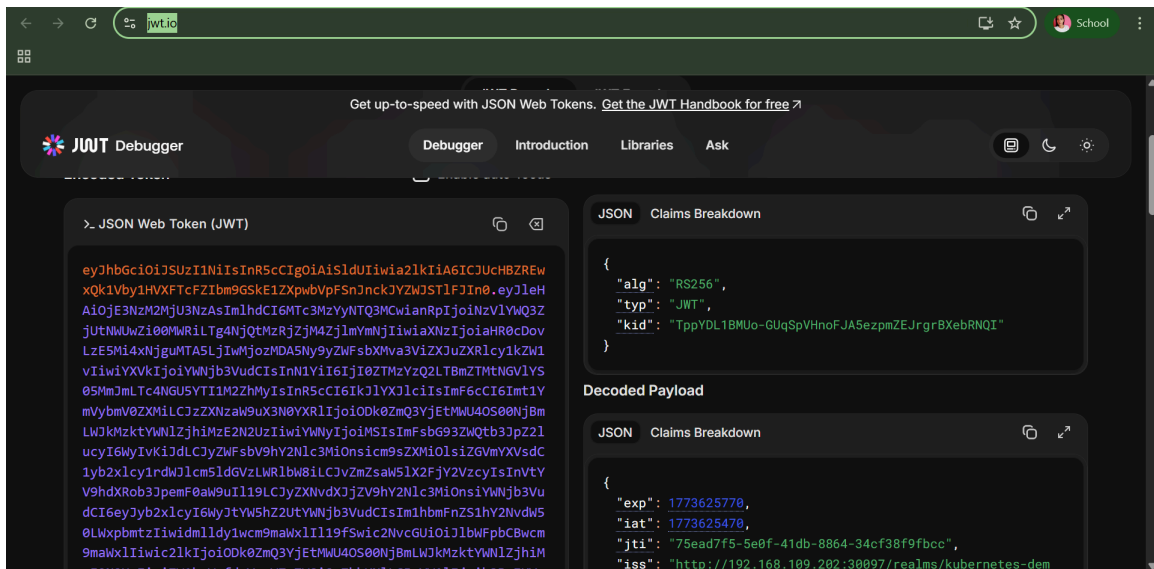
```
curl -X POST
  "http://192.168.109.202:30097/realms/kubernetes-demo/protocol/openid-connect/token" \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "client_id=kubernetes" \
-d "username=ops-user" \
-d "password=Ops1234!" \
-d "grant_type=password"
```

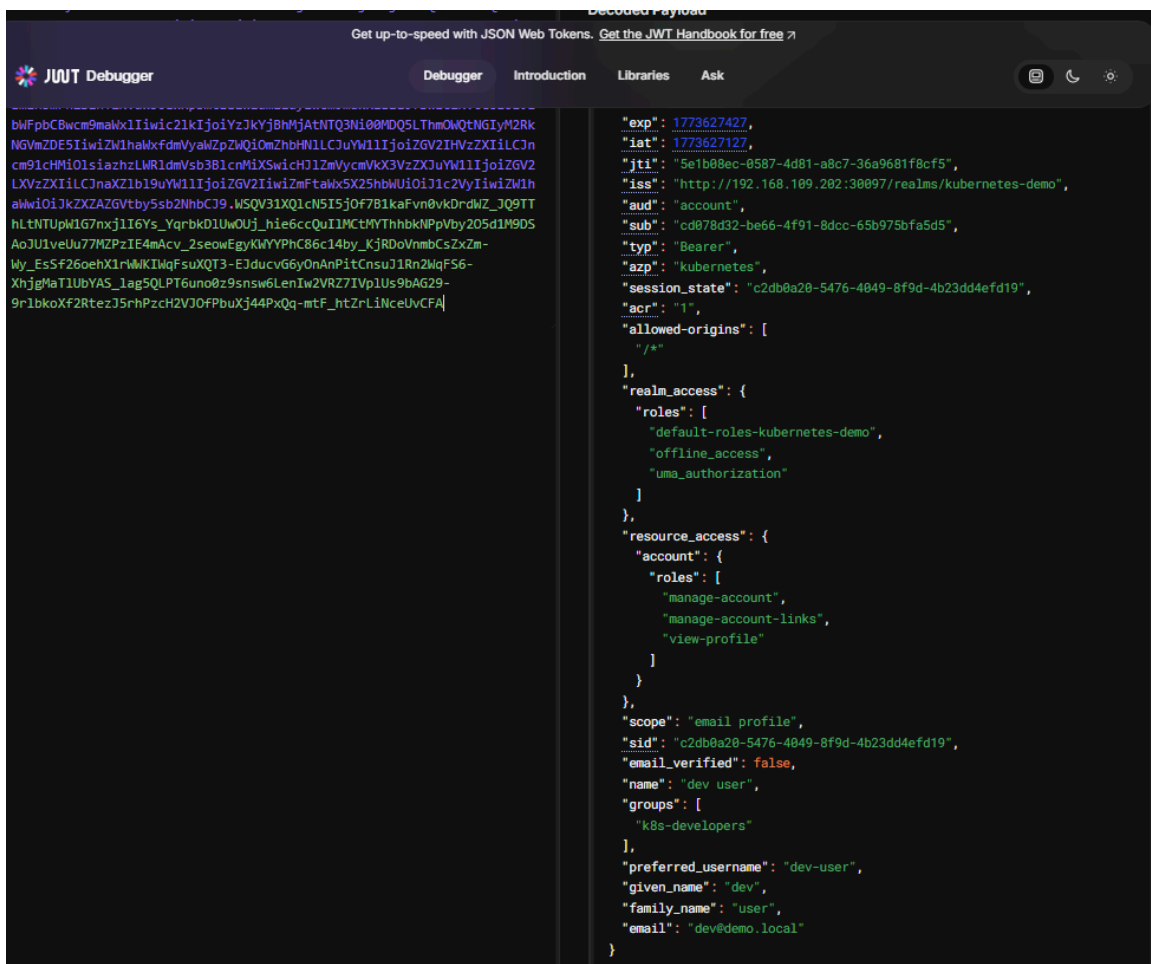
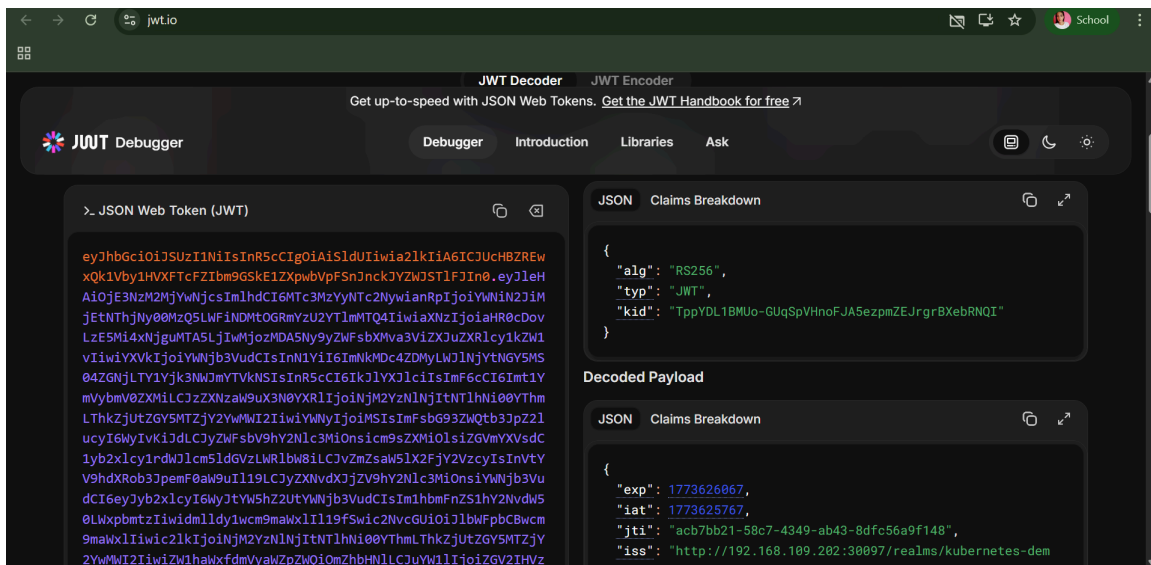
Usuario dev-user

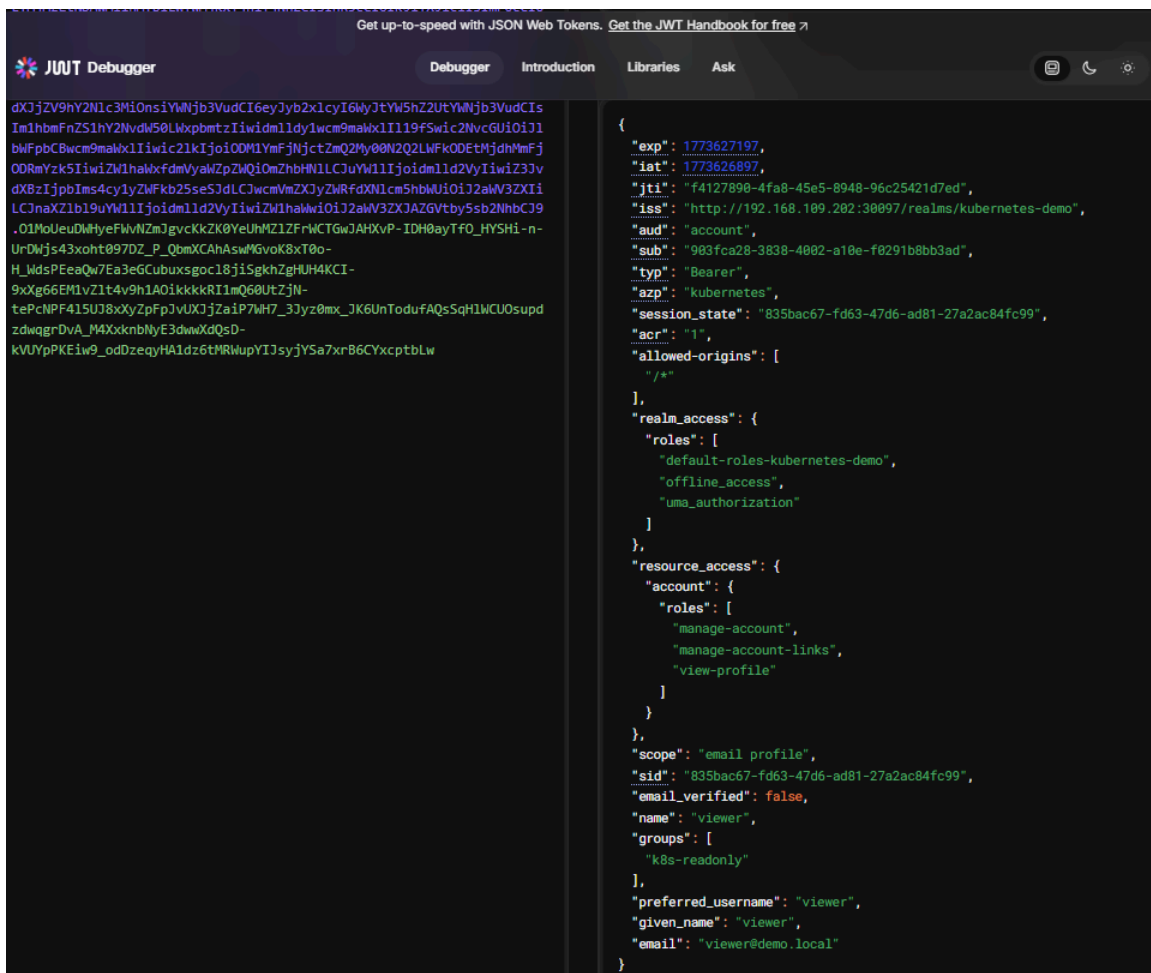
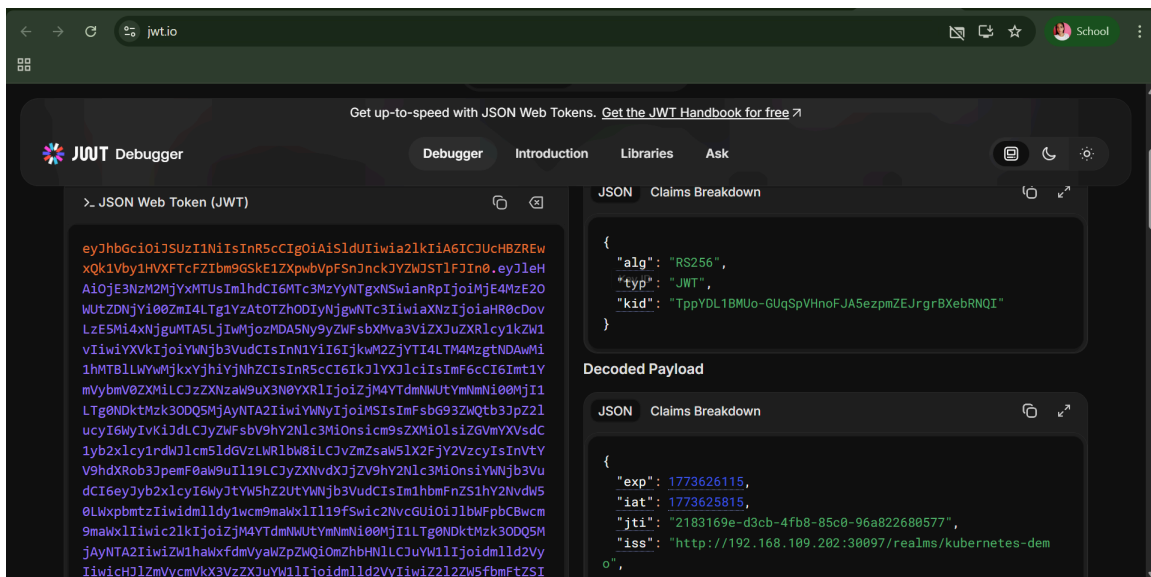
```
curl -X POST
  "http://192.168.109.202:30097/realms/kubernetes-demo/protocol/openid-connect/token" \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "client_id=kubernetes" \
-d "username=dev-user" \
-d "password=Dev1234!" \
-d "grant_type=password"
```

Usuario viewer

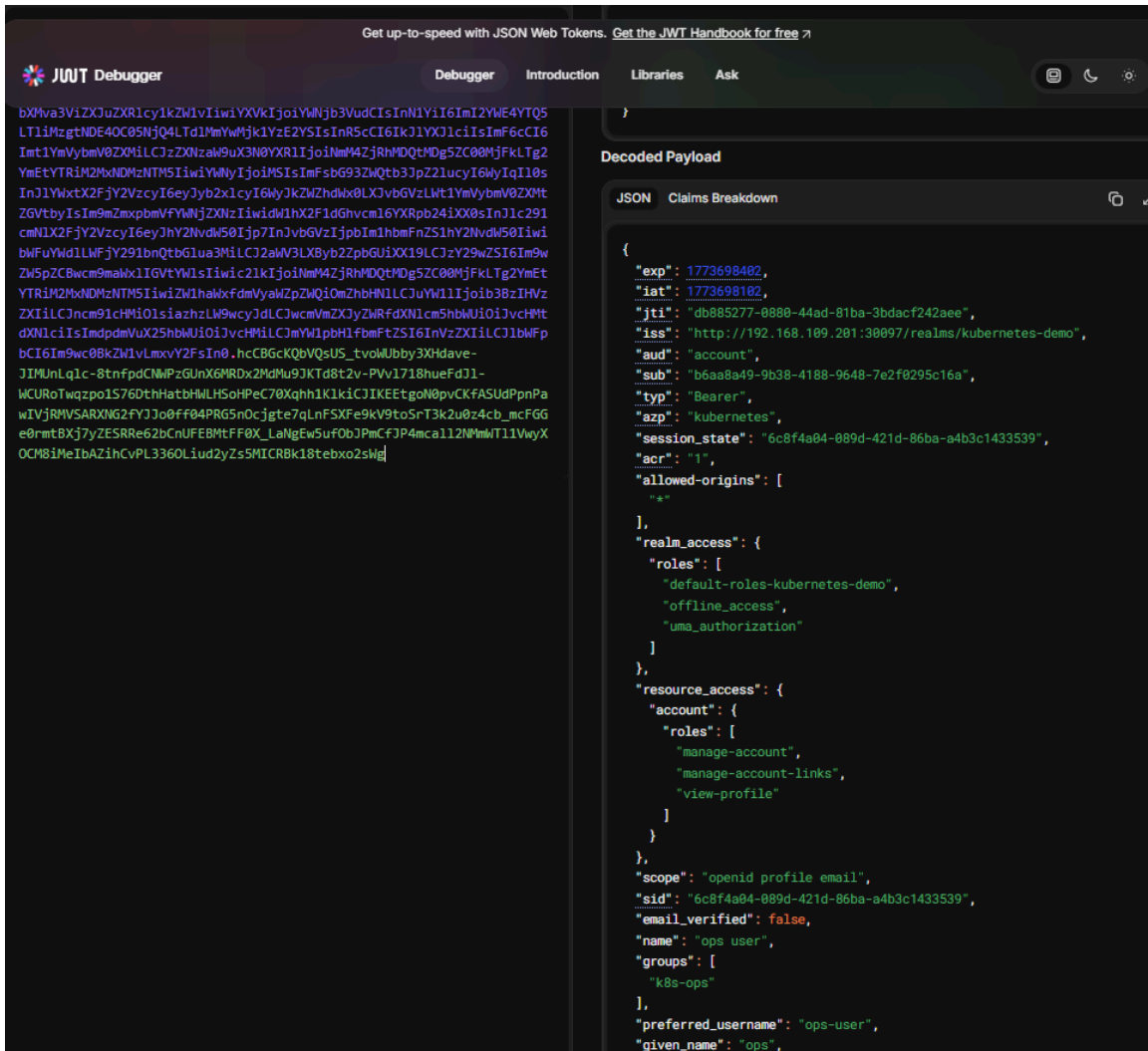
```
curl -X POST
  "http://192.168.109.202:30097/realms/kubernetes-demo/protocol/openid-connect/token" \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "client_id=kubernetes" \
-d "username=viewer" \
-d "password=View1234!" \
-d "grant_type=password"
```





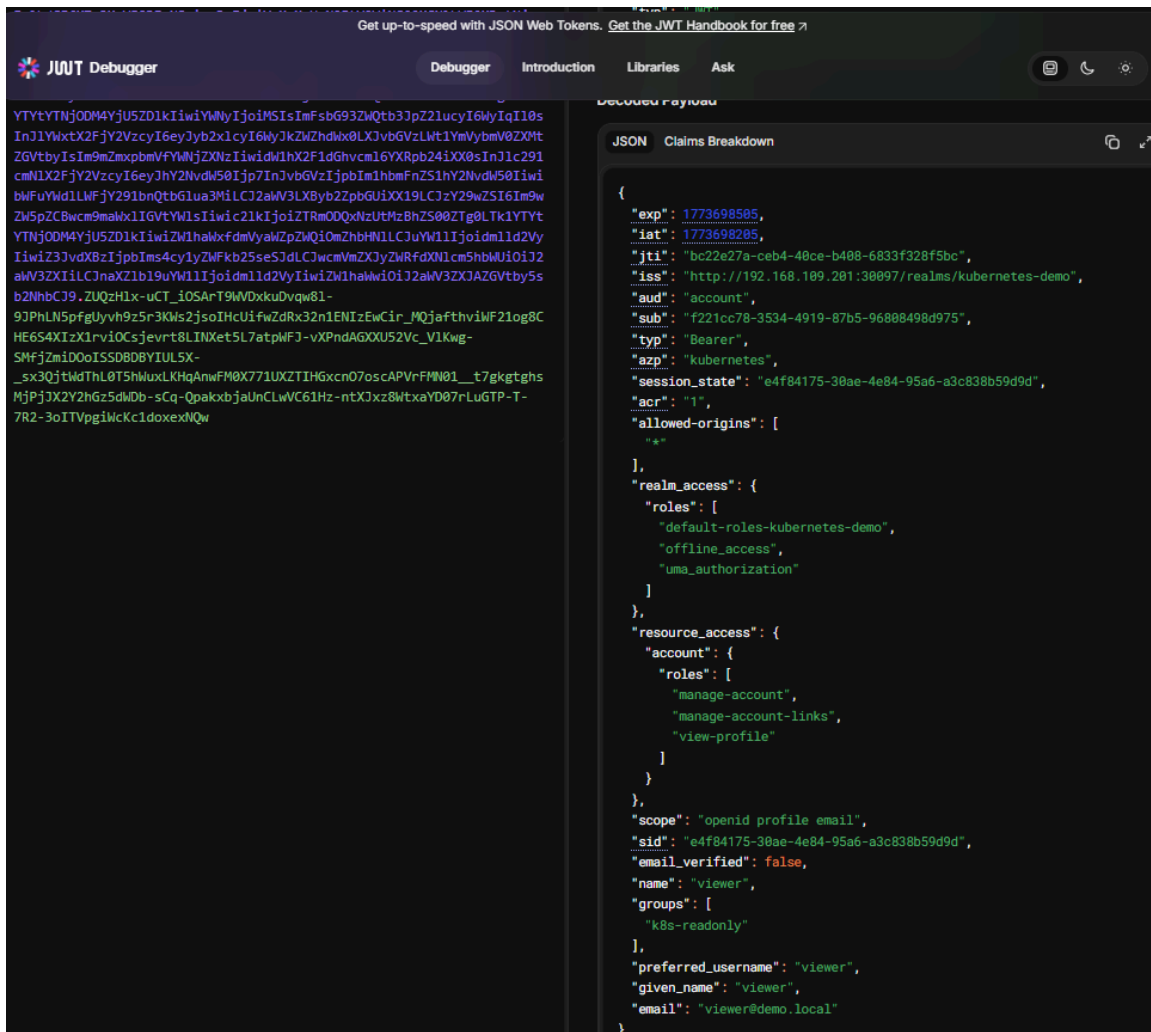
[view](#)

[illegible]



```
curl -X POST
http://192.168.109.201:30097/realms/kubernetes-demo/protocol/openid-connect/token -d
"grant_type=password" -d "client_id=kubernetes" -d
"client_secret=wdjTPGGpodak7lyTc2wqpT5qldTzbBI8" -d "username=viewer" -d
"password=View1234!" -d "scope=openid"
```

[illegible]



Fase 4: RBAC mapeado a grupos de Keycloak

```
[root@k8s-master01 Modulo04]# vi 01-rbac-grupos.yaml
```

```
kubectl apply -f 01-rbac-grupos.yaml
```

```
[root@k8s-master01 Modulo04]# kubectl apply -f 01-rbac-grupos.yaml
clusterrolebinding.rbac.authorization.k8s.io/keycloak-ops-admin created
namespace/proyecto-dev created
rolebinding.rbac.authorization.k8s.io/keycloak-dev-role created
clusterrolebinding.rbac.authorization.k8s.io/keycloak-readonly created
```

Prueba de **ops-user** (Acceso total) Obtén el token de ops-user y mira los nodos

```
# Sacar el token primero
TOKEN_OPS=$(curl -s -X POST
http://192.168.109.201:30097/realms/kubernetes-demo/protocol/openid-connect/token -d
"grant_type=password" -d "client_id=kubernetes" -d
"client_secret=wdjTPGGpodaK7lyTc2wqpT5q1dTzbBI8" -d "username=ops-user" -d
"password=Ops1234!" -d "scope=openid" | jq -r .access_token)
```

```
# ver los nodos sin error
kubectl get nodes --token=$TOKEN_OPS
```

```
[root@k8s-master01 Modulo04]# TOKEN_OPS=$(curl -s -X POST http://192.168.109.201:30097/realms/kubernetes-demo/protocol/openid-connect/token -d "grant_type=password" -d "client_id=kubernetes" -d "client_secret=wdjTPGGpodaK7lyTc2wqpT5q1dTzbBI8" -d "username=ops-user" -d "password=Ops1234!" -d "scope=openid" | jq -r .access_token)
[root@k8s-master01 Modulo04]# kubectl get nodes --token=$TOKEN_OPS
NAME          STATUS    ROLES    AGE   VERSION
k8s-master01  Ready    control-plane   24h   v1.28.15
k8s-worker01  Ready    <none>         23h   v1.28.15
k8s-worker02  Ready    <none>         23h   v1.28.15
```

Prueba de **dev-user** (Solo en su namespace)

```
# Sacar el token
TOKEN_DEV=$(curl -s -X POST
http://192.168.109.201:30097/realms/kubernetes-demo/protocol/openid-connect/token -d
"grant_type=password" -d "client_id=kubernetes" -d
"client_secret=wdjTPGGpodaK7lyTc2wqpT5q1dTzbBI8" -d "username=dev-user" -d
"password=Dev1234!" -d "scope=openid" | jq -r .access_token)
```

```
# Prueba 1: NO debería dejarte ver nodos (es cluster-scope)
kubectl get nodes --token=$TOKEN_DEV
```

```
# Prueba 2: SÍ debería dejarte crear un pod en SU namespace
kubectl run test-pod --image=nginx -n proyecto-dev --token=$TOKEN_DEV
```

```
[root@k8s-master01 Modulo04]# TOKEN_DEV=$(curl -s -X POST http://192.168.109.201:30097/realms/kubernetes-demo/protocol/openid-connect/token -d "grant_type=password" -d "client_id=kubernetes" -d "client_secret=wdjTPGGpodaK7lyTc2wqpT5q1dTzbBI8" -d "username=dev-user" -d "password=Dev1234!" -d "scope=openid" | jq -r .access_token)
[root@k8s-master01 Modulo04]# kubectl get nodes --token=$TOKEN_DEV
NAME          STATUS    ROLES    AGE   VERSION
k8s-master01  Ready    control-plane   24h   v1.28.15
k8s-worker01  Ready    <none>         23h   v1.28.15
k8s-worker02  Ready    <none>         23h   v1.28.15
[root@k8s-master01 Modulo04]# kubectl run test-pod --image=nginx -n proyecto-dev --token=$TOKEN_DEV
pod/test-pod created
```

Prueba de **viewer** (Solo lectura)

```
# Saca el token
TOKEN_VIEW=$(curl -s -X POST
http://192.168.109.201:30097/realms/kubernetes-demo/protocol/openid-connect/token -d
"grant_type=password" -d "client_id=kubernetes" -d
"client_secret=wjTPGGpodaK7lyTc2wqpT5q1dTzbBI8" -d "username=viewer" -d
"password=View1234!" -d "scope=openid" | jq -r .access_token)

# Debería dejarte ver pods, pero NO borrarlos
kubectl get pods -A --token=$TOKEN_VIEW
```

```
[root@k8s-master01 Modulo04]# TOKEN_VIEW=$(curl -s -X POST http://192.168.109.201:30097/realms/kubernetes-demo/protocol/openid-connect/token -d "grant_type=password" -d "client_id=kubernetes" -d "client_secret=wjTPGGpodaK7lyTc2wqpT5q1dTzbBI8" -d "username=viewer" -d "password=View1234!" -d "scope=openid" | jq -r .access_token)
[root@k8s-master01 Modulo04]# kubectl get pods -A --token=$TOKEN_VIEW
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
default	web-test-5bb6bb4cbb-gckn4	1/1	Running	2 (7h10m ago)	23h
default	web-test-5bb6bb4cbb-sq7cq	1/1	Running	2 (7h10m ago)	23h
keycloak	keycloak-7998879487-gx72c	1/1	Running	0	169m
kube-system	calico-kube-controllers-658d97c59c-l6m6r	1/1	Running	37 (49m ago)	24h
kube-system	calico-node-89nj6	1/1	Running	3 (49m ago)	24h
kube-system	calico-node-dv7gk	1/1	Running	2 (7h10m ago)	23h
kube-system	calico-node-sgl5p	1/1	Running	2 (7h10m ago)	23h
kube-system	coredns-5dd5756b68-6gvv7	1/1	Running	3 (49m ago)	24h
kube-system	coredns-5dd5756b68-df4bh	1/1	Running	3 (49m ago)	24h
kube-system	etcd-k8s-master01	1/1	Running	3 (49m ago)	24h
kube-system	kube-apiserver-k8s-master01	1/1	Running	1 (49m ago)	77m
kube-system	kube-controller-manager-k8s-master01	1/1	Running	23 (49m ago)	24h
kube-system	kube-proxy-84wkt	1/1	Running	2 (7h10m ago)	23h
kube-system	kube-proxy-ktcr5	1/1	Running	3 (49m ago)	24h
kube-system	kube-proxy-xj5vz	1/1	Running	2 (7h10m ago)	23h
kube-system	kube-scheduler-k8s-master01	1/1	Running	22 (49m ago)	24h
proyecto-dev	test-pod	1/1	Running	0	2m10s

```
[root@k8s-master01 Modulo04]# kubectl get clusterrolebinding | grep keycloak
keycloak-ops-admin          ClusterRole/cluster-admin
14m
keycloak-readonly          ClusterRole/view
14m
[root@k8s-master01 Modulo04]# kubectl get rolebinding -n proyecto-dev
NAME          ROLE          AGE
keycloak-dev-role ClusterRole/edit 14m
[root@k8s-master01 Modulo04]# kubectl auth can-i create pods \
--as=dev-user \
--as-group=keycloak:k8s-developers \
-n proyecto-dev
no
[root@k8s-master01 Modulo04]# kubectl auth can-i create pods \
--as=dev-user \
--as-group=keycloak:k8s-developers \
-n default
no
```

1. Problema inicial

Keycloak estaba expuesto por HTTP:

<http://192.168.109.201:30097>

Problemas detectados:

- Kubernetes no acepta OIDC con HTTP

- Error en apiserver:

invalid scheme "http", require "https"

2. Solución: Configurar Keycloak con HTTPS

Crear certificado TLS

```
openssl req -x509 -nodes -days 365 \
-newkey rsa:2048 \
-keyout tls.key \
-out tls.crt \
-subj "/CN=keycloak/O=pruebas" \
-addext "subjectAltName=IP:192.168.109.201,DNS:keycloak"
```

Crear secret en Kubernetes

```
kubect1 create secret tls keycloak-tls-secret \
--cert=tls.crt \
--key=tls.key \
-n keycloak
```

Modificar deployment de Keycloak

Cambios importantes:

- ```
env:
```
- name: KC\_HTTP\_ENABLED  
value: "false"
  - name: KC\_HTTPS\_PORT  
value: "8443"
  - name: KC\_HTTPS\_CERTIFICATE\_FILE  
value: /opt/keycloak/certs/tls.crt
  - name: KC\_HTTPS\_CERTIFICATE\_KEY\_FILE  
value: /opt/keycloak/certs/tls.key
- 

### Exponer por NodePort HTTPS

```
ports:
• port: 8443

targetPort: 8443

nodePort: 30443

name: https
type: NodePort
```

---

## Resultado

Cambio clave:

- http://192.168.109.201:30097
- + https://192.168.109.201:30443
- 

## 3. Validar Keycloak HTTPS

```
curl --cacert /root/tls.crt \
https://192.168.109.201:30443/realms/kubernetes-demo/.well-known/openid-configuration
```

Resultado esperado: JSON con endpoints OIDC.

---

## 4. Configurar kube-apiserver

Editar:

```
/etc/kubernetes/manifests/kube-apiserver.yaml
```

Agregar/modificar:

- oidc-issuer-url=https://192.168.109.201:30443/realms/kubernetes-demo
  - oidc-client-id=kubernetes
  - oidc-username-claim=preferred\_username
  - oidc-groups-claim=groups
  - oidc-groups-prefix=keycloak-
  - oidc-ca-file=/etc/kubernetes/pki/keycloak-ca.crt
- 

## Problema crítico encontrado

Existían backups en:

```
/etc/kubernetes/manifests/
```

Ejemplo:

```
kube-apiserver.yaml.bak.*
```

Estos archivos eran leídos por kubelet y causaban conflicto.

---

## Solución

```
mkdir -p /root/k8s-manifest-backups
```

```
mv /etc/kubernetes/manifests/kube-apiserver.yaml.bak.* \
/root/k8s-manifest-backups/
```

---

### Reiniciar kubelet

```
systemctl restart kubelet
```

---

### Validar apiserver

```
kubectl get nodes
```

Resultado:

Ready

---

## 5. Configurar kubectl con OIDC

### Instalar plugin

```
kubectl krew install oidc-login
```

---

### Configurar usuario

```
kubectl config set-credentials dev-user \
 --exec-api-version=client.authentication.k8s.io/v1beta1 \
 --exec-command=kubectl \
 --exec-arg=oidc-login \
 --exec-arg=get-token \
 --exec-arg=--grant-type=device-code \
 --exec-arg=--oidc-issuer-url=https://192.168.109.201:30443/realms/kubernetes-demo \
 --exec-arg=--oidc-client-id=kubernetes \
 --exec-arg=--certificate-authority=/etc/kubernetes/pki/keycloak-ca.crt \
 --exec-arg=--oidc-extra-scope=groups
```

---

### Crear contexto

```
kubectl config set-context dev-context \
 --cluster=kubernetes \
 --user=dev-user \
 --namespace=proyecto-dev
```

---

## 6. Autenticación (Device Code)

```
kubectl get pods -n proyecto-dev
```

Salida:

Please visit:

[https://.../device?user\\_code=XXXX](https://.../device?user_code=XXXX)

Abrir en navegador e iniciar sesión.

---

## 7. Configurar RBAC

### Crear Role + RoleBinding

```

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 name: dev-pod-reader
 namespace: proyecto-dev
rules:
 • apiGroups: [""]
 resources: ["pods","services","configmaps"]
 verbs: ["get","list","watch"]

```

```

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
 name: dev-pod-reader-binding
 namespace: proyecto-dev
subjects:
 • kind: Group
 name: keycloak-k8s-developers
apiGroup: rbac.authorization.k8s.io

roleRef:
 kind: Role
 name: dev-pod-reader
apiGroup: rbac.authorization.k8s.io

```

---

## 8. Validación final

### Acceso permitido

```
kubect1 get pods -n proyecto-dev
```

Resultado:

test-pod Running

---

### Acceso denegado

```
kubect1 get pods -n keycloak
```

Resultado:

Error from server (Forbidden)

```
[root@k8s-master01 tmp.kC2hIK0Moe]# kubectl get pods -n proyecto-dev
NAME READY STATUS RESTARTS AGE
test-pod 1/1 Running 1 (99m ago) 29h

[root@k8s-master01 tmp.kC2hIK0Moe]# kubectl get pods -n keycloak
Error from server (Forbidden): pods is forbidden: User "https://192.168.109.201:30443/realms/kubernet.es-demo/dev-user" cannot list resource "pods" in API group "" in the namespace "keycloak"
[root@k8s-master01 tmp.kC2hIK0Moe]#
```

\*Entregables del módulo:\*\*

- `kubectl config get-contexts` mostrando el contexto OIDC
- Demo funcional: login con dev-user en Keycloak → token → kubectl
- Evidencia de los 3 usuarios con diferentes niveles de acceso

Keycloak funcionando (HTTPS)    OIDC está activo en HTTPS

```
[root@k8s-master01 tmp.k2cH1KOMoe# curl --cacert /root/tls.crt \
https://192.168.109.201:30443/realms/kubernetes-demo/known/openid-configuration | head
```

| % Total | % Received | % Xferd  | Average | Speed  | Time  | Time  | Time  | Current |
|---------|------------|----------|---------|--------|-------|-------|-------|---------|
|         |            |          | Dload   | Upload | Total | Spent | Left  | Speed   |
| 100     | 6234       | 100 6234 | 0       | 0      | 69266 | 0     | ----- | 70840   |

```
{
 "issuer": "https://192.168.109.201:30443/realms/kubernetes-demo",
 "authorization_endpoint": "https://192.168.109.201:30443/realms/kubernetes-demo/protocol/openid-connect/auth",
 "token_endpoint": "https://192.168.109.201:30443/realms/kubernetes-demo/protocol/openid-connect/token",
 "introspection_endpoint": "https://192.168.109.201:30443/realms/kubernetes-demo/protocol/openid-connect/token/introspect",
 "userinfo_endpoint": "https://192.168.109.201:30443/realms/kubernetes-demo/protocol/openid-connect/userinfo",
 "end_session_endpoint": "https://192.168.109.201:30443/realms/kubernetes-demo/protocol/openid-connect/logout",
 "frontchannel_logout_session_supported": true,
 "frontchannel_logout_supported": true,
 "jwks_uri": "https://192.168.109.201:30443/realms/kubernetes-demo/protocol/openid-connect/certs",
 "check_session_iframe": "https://192.168.109.201:30443/realms/kubernetes-demo/protocol/openid-connect/login-status-iframe.html",
 "grant_types_supported": [
 "authorization_code",
 "implicit",
 "refresh_token",
 "password",
 "client_credentials",
 "urn:openid:params:grant-type:ciba",
 "urn:ietf:params:oauth:grant-type:device_code",
 "acr_values_supported": [
 "0",
 "1"
],
 "response_types_supported": [
 "code",
 "none",
 "id_token",
 "token",
 "id_token token",
 "code id_token",
 "code token",
 "code id token token",
 "subject_types_supported": [
 "public",
 "pairwise"
],
 "id_token_signing_alg_values_supported": [
 "PS384",
 "ES384",
 "RS384",
 "HS256",
 "HS512",
 "ES256",
 "RS256",
 "HS384",
 "ES512",
 "PS256",
 "PS512",
 "RS512"
],
 "id_token_encryption_alg_values_supported": [
 "RSA-OAEP",
 "RSA-OAEP-256",
 "RSA1_5"
],
 "id_token_encryption_enc_values_supported": [
 "A256GCM",
 "A128GCM",
 "A128CBC-HS256",
 "A192CBC-HS384",
 "A256CBC-HS512"
],
 "userinfo_signing_alg_values_supported": [
 "PS384",
 "ES384",
 "RS384",
 "HS256",
 "HS512",
 "ES256",
 "RS256",
 "HS384",
 "ES512",
 "PS256",
 "PS512",
 "RS512",
 "none"
],
 "userinfo_encryption_alg_values_supported": [
 "RSA-OAEP",
 "RSA-OAEP-256",
 "RSA1_5"
],
 "userinfo_encryption_enc_values_supported": [
 "A256GCM",
 "A128GCM",
 "A128CBC-HS256",
 "A192CBC-HS384",
 "A256CBC-HS512"
],
 "request_object_signing_alg_values_supported": [
 "PS384",
 "ES384",
 "RS384",
 "HS256",
 "HS512",
 "ES256",
 "RS256",
 "HS384",
 "ES512",
 "PS256",
 "PS512",
 "RS512",
 "none"
],
 "request_object_encryption_alg_values_supported": [
 "RSA-OAEP",
 "RSA-OAEP-256",
 "RSA1_5"
],
 "request_object_encryption_enc_values_supported": [
 "A256GCM",
 "A128GCM",
 "A128CBC-HS256",
 "A192CBC-HS384",
 "A256CBC-HS512"
],
 "response_modes_supported": [
 "query",
 "fragment",
 "form_post",
 "query.jwt",
 "fragment.jwt",
 "form_post.jwt",
 "jwt"
],
 "registration_endpoint": "https://192.168.109.201:30443/realms/kubernetes-demo/clients-registrations/openid-connect",
 "token_endpoint_auth_methods_supported": [
 "private_key_jwt",
 "client_secret_basic",
 "client_secret_post",
 "tls_client_auth",
 "client_secret_jwt",
 "token_endpoint_auth_signing_alg_values_supported": [
 "PS384",
 "ES384",
 "RS384",
 "HS256",
 "HS512",
 "ES256",
 "RS256",
 "HS384",
 "ES512",
 "PS256",
 "PS512",
 "RS512"
],
 "introspection_endpoint_auth_methods_supported": [
 "private_key_jwt",
 "client_secret_basic",
 "client_secret_post",
 "tls_client_auth",
 "client_secret_jwt",
 "introspection_endpoint_auth_signing_alg_values_supported": [
 "PS384",
 "ES384",
 "RS384",
 "HS256",
 "HS512",
 "ES256",
 "RS256",
 "HS384",
 "ES512",
 "PS256",
 "PS512",
 "RS512"
],
 "authorization_signing_alg_values_supported": [
 "PS384",
 "ES384",
 "RS384",
 "HS256",
 "HS512",
 "ES256",
 "RS256",
 "HS384",
 "ES512",
 "PS256",
 "PS512",
 "RS512"
],
 "authorization_encryption_alg_values_supported": [
 "RSA-OAEP",
 "RSA-OAEP-256",
 "RSA1_5"
],
 "authorization_encryption_enc_values_supported": [
 "A256GCM",
 "A128GCM",
 "A128CBC-HS256",
 "A192CBC-HS384",
 "A256CBC-HS512"
]
 }
}
```

## Configuración del kube-apiserver

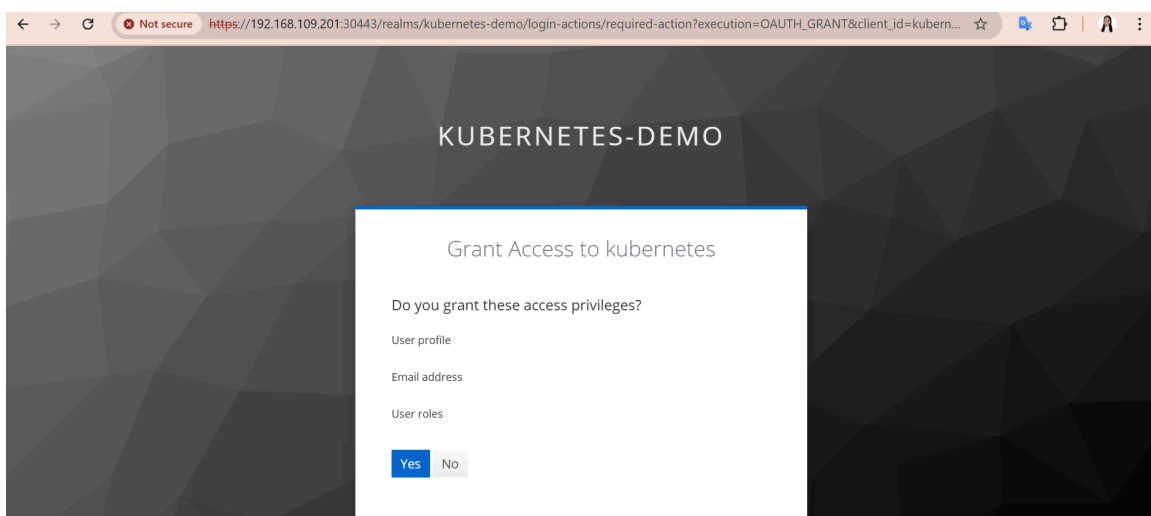
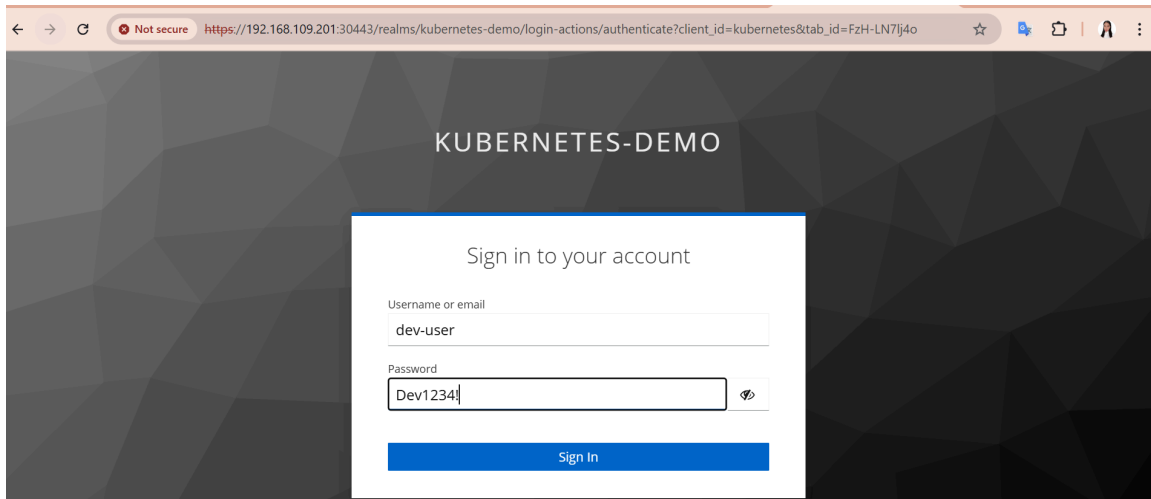
```
[root@k8s-master01 tmp.kC2h1K0Moe]# grep oidc /etc/kubernetes/manifests/kube-apiserver.yaml
- --oidc-issuer-url=https://192.168.109.201:30443/realms/kubernetes-demo
- --oidc-client-id=kubernetes
- --oidc-username-claim=preferred_username
- --oidc-groups-claim=groups
- --oidc-groups-prefix=keycloak-
- --oidc-ca-file=/etc/kubernetes/pki/keycloak-ca.crt
```

## Cluster funcionando

El usuario autenticado mediante OIDC no cuenta con permisos a nivel de cluster (nodes), lo cual confirma que la autorización RBAC está correctamente configurada.

```
[root@k8s-master01 tmp.kC2hIK0Moe]# kubectl get nodes
error: could not open the browser: exec: "xdg-open,x-www-browser,www-browser": executable file not found in $PATH

Please visit the following URL in your browser manually: https://192.168.109.201:30443/realms/kubernetes-demo/device?user_code=SAUB-QVHI
```



```
[root@k8s-master01 tmp.kC2hIKOMoe]# kubectl get nodes
Error: could not open the browser: exec: "xdg-open,x-www-browser,www-browser": executable file not found in $PATH

Please visit the following URL in your browser manually: https://192.168.109.201:30443/realms/kubernetes-demo/device?user_code=SAUB-LQVHI
Error from server (Forbidden): nodes is forbidden: User "https://192.168.109.201:30443/realms/kubernetes-demo#dev-user" cannot list resource "nodes" in API group "" at the cluster scope
[root@k8s-master01 tmp.kC2hIKOMoe]#
```

## Contexto OIDC configurado

```
[root@k8s-master01 tmp.kC2hIKOMoe]# kubectl config get-contexts
CURRENT NAME CLUSTER AUTHINFO NAMESPACE
* dev-context kubernetes dev-user proyecto-dev
 kubernetes-admin@kubernetes kubernetes kubernetes-admin
```

## Autenticación exitosa

```
[root@k8s-master01 tmp.kC2hIKOMoe]# kubectl get pods -n proyecto-dev
NAME READY STATUS RESTARTS AGE
test-pod 1/1 Running 1 (139m ago) 30h
[root@k8s-master01 tmp.kC2hIKOMoe]#
```



## Autorización restringida

```
[root@k8s-master01 tmp.kC2hIKOMoe]# kubectl get pods -n keycloak
Error from server (Forbidden): pods is forbidden: User "https://192.168.109.201:30443/realm/kubernetes-demo#dev-user" cannot list
resource "pods" in API group "" in the namespace "keycloak"
[root@k8s-master01 tmp.kC2hIKOMoe]#
```

## Validación con can-i

```
[root@k8s-master01 tmp.kC2hIKOMoe]# kubectl auth can-i get pods -n proyecto-dev
kubectl auth can-i get pods -n keycloak
yes
no
```

Módulo 5: Se implementó autenticación mediante OIDC con Keycloak usando HTTPS, integrando Kubernetes con un proveedor externo de identidad. Se validó el control de acceso mediante RBAC, permitiendo acceso únicamente al namespace autorizado y restringiendo otros, comprobado mediante respuestas "Forbidden".

## Fase 6: Cambio de grupo en Keycloak = cambio de permisos en Kubernetes

Escenario:

- Keycloak en: `https://192.168.109.201:30443`
  - Realm: `kubernetes-demo`
  - Usuario: `dev-user`
  - Grupo actual: `k8s-developers`
  - Prefijo en kube-apiserver: `keycloak-`
  - Kubernetes ve: `keycloak-k8s-developers`

### Validación de permisos actuales

#### Permitido:

```
kubect1 config use-context dev-context
rm -rf ~/.kube/cache/oidc-login
```

#### Restringido:

```
kubect1 get pods -n proyecto-dev
kubect1 get nodes
```

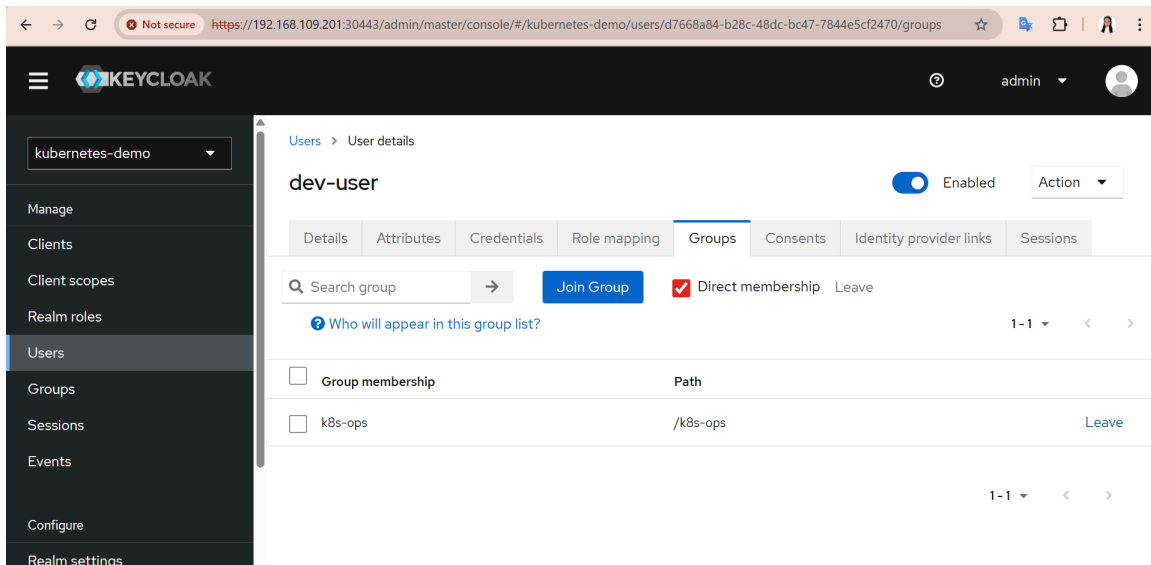
```
[root@k8s-master01 ~]# kubect1 config use-context dev-context
rm -rf ~/.kube/cache/oidc-login
Switched to context "dev-context".
[root@k8s-master01 ~]# kubect1 get pods -n proyecto-dev
kubect1 get nodes
error: could not open the browser: exec: "xdg-open,x-www-browser,www-browser": executable file not found in $PATH

Please visit the following URL in your browser manually: https://192.168.109.201:30443/realms/kubernetes-demo/device?user_code=PXHI-GXMB
NAME READY STATUS RESTARTS AGE
test-pod 1/1 Running 1 (158m ago) 30h
Error from server (Forbidden): nodes is forbidden: User "https://192.168.109.201:30443/realms/kubernetes-demo#dev-user" cannot list resource "nodes" in API group "" at the cluster scope
```

### Configurar permisos para k8s-ops (cluster-admin)

```
[root@k8s-master01 ~]# kubect1 config use-context kubernetes-admin@kubernetes
Switched to context "kubernetes-admin@kubernetes".
[root@k8s-master01 ~]# cat > /root/k8s-ops-admin.yaml <<'EOF'
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
 name: keycloak-k8s-ops-clusteradmin
subjects:
- kind: Group
 name: keycloak-k8s-ops
 apiGroup: rbac.authorization.k8s.io
roleRef:
 kind: ClusterRole
 name: cluster-admin
 apiGroup: rbac.authorization.k8s.io
EOF
[root@k8s-master01 ~]# kubect1 apply -f /root/k8s-ops-admin.yaml
clusterrolebinding.rbac.authorization.k8s.io/keycloak-k8s-ops-clusteradmin created
```

## Cambiar grupo en Keycloak



## Forzar nuevo token OIDC y Verificar nuevos permisos

```
[root@k8s-master01 ~]# kubectl config use-context dev-context
rm -rf ~/.kube/cache/oidc-login
Switched to context "dev-context".
[root@k8s-master01 ~]# kubectl get nodes
error: could not open the browser: exec: "xdg-open,x-www-browser,www-browser": executable file not found in $PATH

Please visit the following URL in your browser manually: https://192.168.109.201:30443/realms/kubernetes-demo/device?user_code=WDFA-I00J

NAME STATUS ROLES AGE VERSION
k8s-master01 Ready control-plane 2d7h v1.28.15
k8s-worker01 Ready <none> 2d6h v1.28.15
k8s-worker02 Ready <none> 2d6h v1.28.15
[root@k8s-master01 ~]# kubectl get nodes
NAME STATUS ROLES AGE VERSION
k8s-master01 Ready control-plane 2d7h v1.28.15
k8s-worker01 Ready <none> 2d6h v1.28.15
k8s-worker02 Ready <none> 2d6h v1.28.15
```

### Módulo 6:

Inicialmente el usuario **dev-user** pertenecía al grupo **k8s-developers**, lo que limitaba su acceso únicamente al namespace **proyecto-dev**. Al intentar listar los nodos del clúster, se obtenía un error de tipo "Forbidden". Posteriormente, el usuario fue movido al grupo **k8s-ops** en Keycloak, el cual está asociado al rol **cluster-admin** en Kubernetes. Tras forzar la obtención de un nuevo token OIDC, el usuario pudo listar los nodos del clúster exitosamente, demostrando que los permisos en Kubernetes se asignan dinámicamente en función de los grupos definidos en Keycloak.

## Módulo 7 — Observabilidad: Jaeger + Fluent Bit + Loki

Instalamos la herramienta helm

```
[root@k8s-master01 ~]# cd /tmp
[root@k8s-master01 tmp]# curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
Downloading https://get.helm.sh/helm-v3.20.1-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
[root@k8s-master01 tmp]#
```

### Verificar namespace

```
[root@k8s-master01 ~]# kubectl get ns
NAME STATUS AGE
default Active 3d4h
keycloak Active 3d1h
kube-node-lease Active 3d4h
kube-public Active 3d4h
kube-system Active 3d4h
logging Active 9h
proyecto-dev Active 2d3h
tracing Active 9h
[root@k8s-master01 ~]#
```

### Validar despliegue de JAEGER

```
[root@k8s-master01 ~]# kubectl get pods -n tracing
NAME READY STATUS RESTARTS AGE
jaeger-857965475c-mpdz5 1/1 Running 0 9h
```

### Servicios

```
[root@k8s-master01 ~]# kubectl get svc -n tracing
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
jaeger-collector ClusterIP 10.105.139.160 <none> 4317/TCP,4318/TCP,14268/TCP 9h
jaeger-ui NodePort 10.103.213.127 <none> 16686:30100/TCP 9h
```

## Tráfico

```
[root@k8s-master01 ~]# kubectl get pods -A
```

| NAMESPACE    | NAME                                     | READY | STATUS  | RESTARTS     | AGE   |
|--------------|------------------------------------------|-------|---------|--------------|-------|
| default      | net-b                                    | 1/1   | Running | 11 (40m ago) | 20h   |
| default      | web-test-5bb6bb4cbb-25t9w                | 1/1   | Running | 0            | 149m  |
| default      | web-test-5bb6bb4cbb-gckn4                | 1/1   | Running | 4 (10h ago)  | 3d3h  |
| keycloak     | keycloak-6bdf94948-7klmf                 | 1/1   | Running | 0            | 149m  |
| kube-system  | calico-kube-controllers-658d97c59c-ldm6r | 1/1   | Running | 48 (10h ago) | 3d4h  |
| kube-system  | calico-node-p87s6                        | 1/1   | Running | 0            | 9h    |
| kube-system  | calico-node-pks89                        | 1/1   | Running | 0            | 9h    |
| kube-system  | calico-node-vzrsk                        | 1/1   | Running | 0            | 9h    |
| kube-system  | coredns-5dd5756b68-6gvv7                 | 1/1   | Running | 5 (10h ago)  | 3d4h  |
| kube-system  | coredns-5dd5756b68-df4bh                 | 1/1   | Running | 5 (10h ago)  | 3d4h  |
| kube-system  | etcd-k8s-master01                        | 1/1   | Running | 5 (10h ago)  | 3d4h  |
| kube-system  | kube-apiserver-k8s-master01              | 1/1   | Running | 1 (10h ago)  | 22h   |
| kube-system  | kube-controller-manager-k8s-master01     | 1/1   | Running | 49 (48m ago) | 3d4h  |
| kube-system  | kube-proxy-84wkt                         | 1/1   | Running | 4 (10h ago)  | 3d3h  |
| kube-system  | kube-proxy-ktcr5                         | 1/1   | Running | 5 (10h ago)  | 3d4h  |
| kube-system  | kube-proxy-xj5vz                         | 1/1   | Running | 4 (10h ago)  | 3d3h  |
| kube-system  | kube-scheduler-k8s-master01              | 1/1   | Running | 47 (31m ago) | 3d4h  |
| logging      | loki-0                                   | 1/1   | Running | 0            | 138m  |
| logging      | loki-fluent-bit-loki-bpbs6               | 1/1   | Running | 0            | 6h20m |
| logging      | loki-grafana-86b4946968-79sb5            | 2/2   | Running | 0            | 149m  |
| logging      | loki-promtail-vz9fd                      | 1/1   | Running | 0            | 4h47m |
| proyecto-dev | test-pod                                 | 1/1   | Running | 2 (10h ago)  | 2d3h  |
| tracing      | jaeger-857965475c-mpdz5                  | 1/1   | Running | 0            | 9h    |

### Fluent Bit

### DaemonSet

```
[root@k8s-master01 ~]# kubectl get ds -n logging
```

| NAME                 | DESIRED | CURRENT | READY | UP-TO-DATE | AVAILABLE | NODE SELECTOR        | AGE |
|----------------------|---------|---------|-------|------------|-----------|----------------------|-----|
| loki-fluent-bit-loki | 1       | 1       | 1     | 1          | 1         | logging-node=enabled | 9h  |
| loki-promtail        | 1       | 1       | 1     | 1          | 1         | logging-node=enabled | 9h  |

## Pods

```
[root@k8s-master01 ~]# kubectl get pods -n logging
```

| NAME                          | READY | STATUS  | RESTARTS | AGE   |
|-------------------------------|-------|---------|----------|-------|
| loki-0                        | 1/1   | Running | 0        | 141m  |
| loki-fluent-bit-loki-bpbs6    | 1/1   | Running | 0        | 6h23m |
| loki-grafana-86b4946968-79sb5 | 2/2   | Running | 0        | 152m  |
| loki-promtail-vz9fd           | 1/1   | Running | 0        | 4h50m |

```
[root@k8s-master01 ~]# kubectl logs -n logging loki-fluent-bit-loki-bpbs6
```

```
Fluent Bit v1.4.6
* Copyright (C) 2019-2020 The Fluent Bit Authors
* Copyright (C) 2015-2018 Treasure Data
* Fluent Bit is a CNCF sub-project under the umbrella of Fluentd
* https://fluentbit.io

[2026/03/18 19:44:09] [info] [storage] version=1.0.3, initializing...
[2026/03/18 19:44:09] [info] [storage] in-memory
[2026/03/18 19:44:09] [info] [storage] normal synchronization mode, checksum disabled, max_chunks_up=128
[2026/03/18 19:44:09] [info] [engine] started (pid=1)
[2026/03/18 19:44:09] [info] [filter:kubernetes:kubernetes.0] https=1 host=kubernetes.default.svc port=443
[2026/03/18 19:44:09] [info] [filter:kubernetes:kubernetes.0] local POD info OK
[2026/03/18 19:44:09] [info] [filter:kubernetes:kubernetes.0] testing connectivity with API server...
[2026/03/18 19:44:09] [info] [filter:kubernetes:kubernetes.0] Wait 30 secs until DNS starts up (1/6)
[2026/03/18 19:44:39] [info] [filter:kubernetes:kubernetes.0] Wait 30 secs until DNS starts up (2/6)
[2026/03/18 19:45:09] [info] [filter:kubernetes:kubernetes.0] Wait 30 secs until DNS starts up (3/6)
[2026/03/18 19:45:39] [info] [filter:kubernetes:kubernetes.0] Wait 30 secs until DNS starts up (4/6)
[2026/03/18 19:46:09] [info] [filter:kubernetes:kubernetes.0] Wait 30 secs until DNS starts up (5/6)
[2026/03/18 19:46:39] [info] [filter:kubernetes:kubernetes.0] Wait 30 secs until DNS starts up (6/6)
[2026/03/18 19:47:09] [warn] [filter:kubernetes:kubernetes.0] could not resolve kubernetes.default.svc
[2026/03/18 19:47:09] [info] [http_server] listen iface=0.0.0.0 tcp_port=2020
[2026/03/18 19:47:09] [info] [sp] stream processor started
[2026/03/18 19:47:09] [warn] net_tcp_fd_connect: getaddrinfo(host='kubernetes.default.svc'): Temporary failure in name resolution
[2026/03/18 19:47:09] [error] [filter:kubernetes:kubernetes.0] upstream connection error
```

## Loki-Pods

```
[root@k8s-master01 ~]# kubectl get pods -n logging | grep loki
```

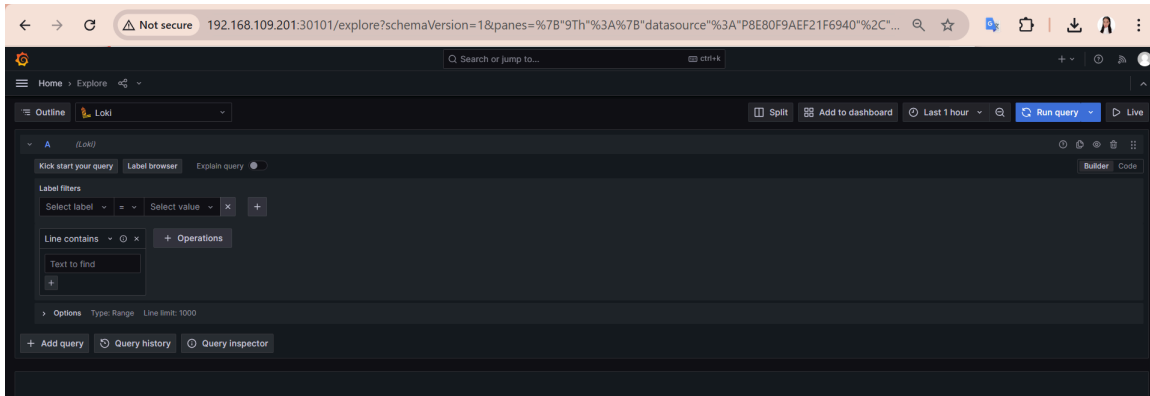
|                               |     |         |   |       |
|-------------------------------|-----|---------|---|-------|
| loki-0                        | 1/1 | Running | 0 | 143m  |
| loki-fluent-bit-loki-bpbs6    | 1/1 | Running | 0 | 6h26m |
| loki-grafana-86b4946968-79sb5 | 2/2 | Running | 0 | 154m  |
| loki-promtail-vz9fd           | 1/1 | Running | 0 | 4h53m |

## Loki servicio activo

## Grafana

```
[root@k8s-master01 ~]# kubectl get svc -n logging
```

| NAME            | TYPE      | CLUSTER-IP    | EXTERNAL-IP | PORT(S)        | AGE |
|-----------------|-----------|---------------|-------------|----------------|-----|
| loki            | NodePort  | 10.101.5.47   | <none>      | 3100:30102/TCP | 9h  |
| loki-grafana    | NodePort  | 10.109.82.119 | <none>      | 80:30101/TCP   | 9h  |
| loki-headless   | ClusterIP | None          | <none>      | 3100/TCP       | 9h  |
| loki-memberlist | ClusterIP | None          | <none>      | 7946/TCP       | 9h  |



```
[root@k8s-master01 ~]# kubectl get svc -n logging
```

| NAME            | TYPE      | CLUSTER-IP    | EXTERNAL-IP | PORT(S)        | AGE |
|-----------------|-----------|---------------|-------------|----------------|-----|
| loki            | NodePort  | 10.101.5.47   | <none>      | 3100:30102/TCP | 9h  |
| loki-grafana    | NodePort  | 10.109.82.119 | <none>      | 80:30101/TCP   | 9h  |
| loki-headless   | ClusterIP | None          | <none>      | 3100/TCP       | 9h  |
| loki-memberlist | ClusterIP | None          | <none>      | 7946/TCP       | 9h  |

```
[root@k8s-master01 ~]# kubectl get svc -n logging
```

| NAME            | TYPE      | CLUSTER-IP    | EXTERNAL-IP | PORT(S)        | AGE |
|-----------------|-----------|---------------|-------------|----------------|-----|
| loki            | NodePort  | 10.101.5.47   | <none>      | 3100:30102/TCP | 9h  |
| loki-grafana    | NodePort  | 10.109.82.119 | <none>      | 80:30101/TCP   | 9h  |
| loki-headless   | ClusterIP | None          | <none>      | 3100/TCP       | 9h  |
| loki-memberlist | ClusterIP | None          | <none>      | 7946/TCP       | 9h  |

```
[root@k8s-master01 ~]# kubectl get all -n logging
```

| NAME                              | READY | STATUS  | RESTARTS | AGE   |
|-----------------------------------|-------|---------|----------|-------|
| pod/loki-0                        | 1/1   | Running | 0        | 147m  |
| pod/loki-fluent-bit-loki-bpbs6    | 1/1   | Running | 0        | 6h30m |
| pod/loki-grafana-86b4946968-79sb5 | 2/2   | Running | 0        | 158m  |
| pod/loki-promtail-vz9fd           | 1/1   | Running | 0        | 4h57m |

| NAME                    | TYPE      | CLUSTER-IP    | EXTERNAL-IP | PORT(S)        | AGE |
|-------------------------|-----------|---------------|-------------|----------------|-----|
| service/loki            | NodePort  | 10.101.5.47   | <none>      | 3100:30102/TCP | 9h  |
| service/loki-grafana    | NodePort  | 10.109.82.119 | <none>      | 80:30101/TCP   | 9h  |
| service/loki-headless   | ClusterIP | None          | <none>      | 3100/TCP       | 9h  |
| service/loki-memberlist | ClusterIP | None          | <none>      | 7946/TCP       | 9h  |

| NAME                                | DESIRED | CURRENT | READY | UP-TO-DATE | AVAILABLE | NODE SELECTOR        | AGE |
|-------------------------------------|---------|---------|-------|------------|-----------|----------------------|-----|
| daemonset.apps/loki-fluent-bit-loki | 1       | 1       | 1     | 1          | 1         | logging-node=enabled | 9h  |
| daemonset.apps/loki-promtail        | 1       | 1       | 1     | 1          | 1         | logging-node=enabled | 9h  |

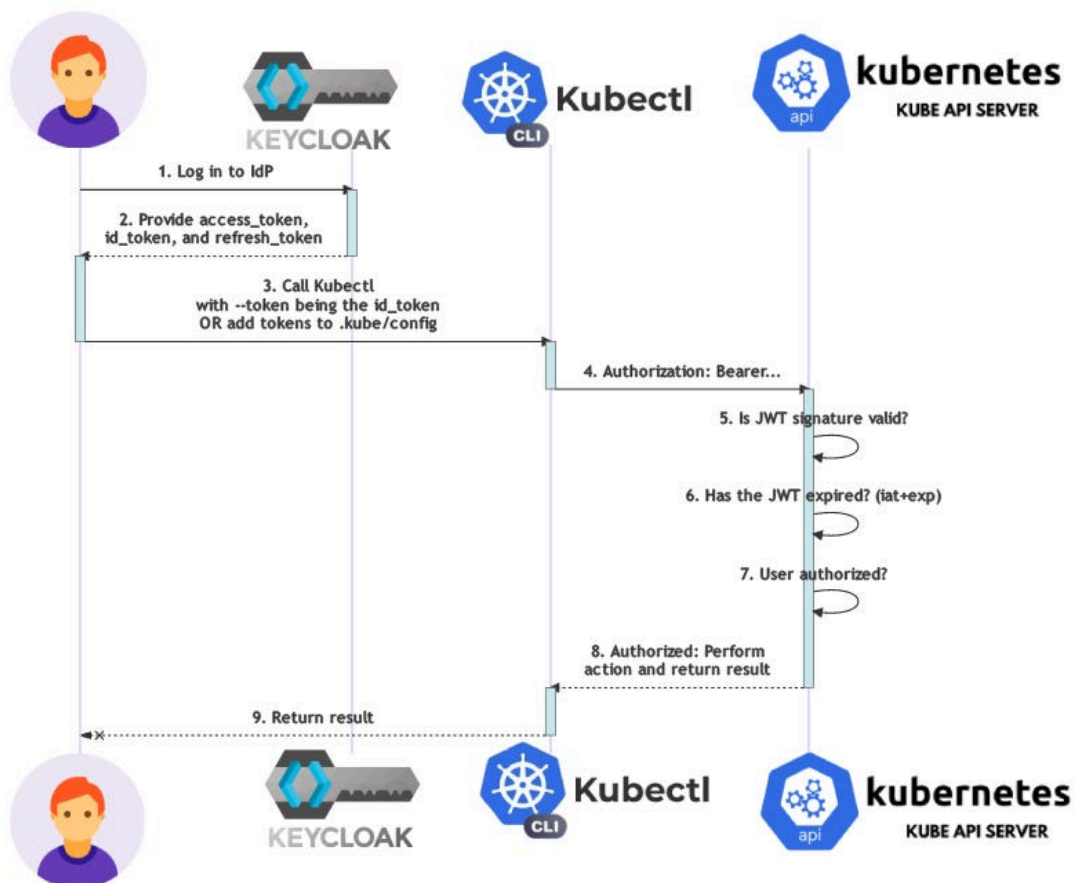
| NAME                         | READY | UP-TO-DATE | AVAILABLE | AGE   |
|------------------------------|-------|------------|-----------|-------|
| deployment.apps/loki-grafana | 1/1   | 1          | 1         | 5h11m |

| NAME                                    | DESIRED | CURRENT | READY | AGE   |
|-----------------------------------------|---------|---------|-------|-------|
| replicaset.apps/loki-grafana-75c5b8d755 | 0       | 0       | 0     | 5h11m |
| replicaset.apps/loki-grafana-86b4946968 | 1       | 1       | 1     | 5h3m  |
| replicaset.apps/loki-grafana-9cb785588  | 0       | 0       | 0     | 5h3m  |

| NAME                  | READY | AGE |
|-----------------------|-------|-----|
| statefulset.apps/loki | 1/1   | 9h  |

## Referencias

- Keycloak: <https://www.keycloak.org/getting-started/getting-started-kube>
- K8s OIDC Auth: <https://kubernetes.io/docs/reference/access-authn-authz/authentication/#openid-connect-tokens>
- kubelogin: <https://github.com/int128/kubelogin>
- Jaeger: <https://www.jaegertracing.io/docs/1.52/operator/>
- Fluent Bit + Loki: <https://grafana.com/docs/loki/latest/send-data/fluentbit/>
- JWT decoder (para verificar tokens): <https://jwt.io>



# Glosario de términos

## A

### **API Server (Kubernetes API Server)**

Componente principal de Kubernetes encargado de recibir solicitudes (kubectl, API REST), validar autenticación y aplicar reglas de autorización mediante RBAC.

---

## C

### **Client (OIDC Client)**

Aplicación registrada en Keycloak que representa a Kubernetes. Permite que el API Server confíe en los tokens emitidos por Keycloak.

### **ClusterRole**

Rol de Kubernetes que define permisos a nivel de todo el clúster.

### **ClusterRoleBinding**

Asociación entre un usuario o grupo y un ClusterRole, otorgando permisos globales en el clúster.

### **Contexto (kubectl context)**

Configuración que define con qué usuario, clúster y namespace se interactúa mediante kubectl.

---

## D

### **DaemonSet**

Recurso de Kubernetes que garantiza que un pod se ejecute en todos los nodos del clúster (ej. Fluent Bit).

### **Deployment**

Objeto de Kubernetes que gestiona la creación y actualización de pods de manera declarativa.

---

## F

### **Fluent Bit**

Agente ligero de logging que recolecta logs de los nodos y los envía a sistemas como Loki.

---

## G

### **Grafana**

Herramienta de visualización que permite consultar y visualizar métricas y logs almacenados en Loki.

### **Grupo (Keycloak Group)**

Entidad en Keycloak que agrupa usuarios y permite asignar permisos de forma colectiva.



## H

### Helm

Gestor de paquetes para Kubernetes que facilita la instalación de aplicaciones mediante charts.

---

## J

### Jaeger

Sistema de trazabilidad distribuida que permite visualizar flujos de ejecución y tiempos de respuesta entre servicios.

### JWT (JSON Web Token)

Token firmado digitalmente que contiene información del usuario (claims), como nombre y grupos, usado para autenticación.

---

## K

### Keycloak

Proveedor de identidad (IdP) que implementa OAuth2 y OpenID Connect para autenticación y autorización centralizada.

#### **kubectl**

Herramienta de línea de comandos para interactuar con Kubernetes.

### kubelogin

Plugin que permite a kubectl autenticarse mediante OIDC usando Keycloak.

### Kubernetes

Plataforma de orquestación de contenedores que automatiza despliegue, escalado y gestión de aplicaciones.

---

## L

### Loki

Sistema de almacenamiento de logs optimizado que indexa metadatos en lugar del contenido completo.

---

## N

### Namespace

Mecanismo de aislamiento lógico dentro de Kubernetes para organizar recursos.

### NodePort

Tipo de servicio en Kubernetes que expone una aplicación en un puerto del nodo.

## O

### **OIDC (OpenID Connect)**

Protocolo de autenticación basado en OAuth2 que permite verificar la identidad de un usuario mediante tokens.

### **OAuth2**

Protocolo de autorización que permite a aplicaciones acceder a recursos en nombre de un usuario.

---

## P

### **Pod**

Unidad básica de ejecución en Kubernetes que contiene uno o más contenedores.

### **Promtail**

Agente que envía logs a Loki, similar a Fluent Bit.

---

## R

### **RBAC (Role-Based Access Control)**

Modelo de control de acceso basado en roles que define qué acciones puede realizar un usuario o grupo.

### **Realm (Keycloak Realm)**

Espacio lógico en Keycloak que contiene usuarios, clientes, roles y configuraciones de autenticación.

### **Role**

Conjunto de permisos definido dentro de un namespace en Kubernetes.

### **RoleBinding**

Asociación entre un usuario o grupo y un Role dentro de un namespace específico.

---

## S

### **Secret (Kubernetes Secret)**

Objeto que almacena información sensible como contraseñas o tokens.

### **Service (Kubernetes Service)**

Recurso que expone un conjunto de pods mediante una dirección IP y un puerto.

---

## T

### **Token**

Credencial digital utilizada para autenticar a un usuario en un sistema.

### **Tracing (Trazabilidad)**

Técnica que permite seguir el flujo de una petición entre múltiples servicios.

---

## **U**

### **Usuario (Keycloak User)**

Entidad que representa a una persona o sistema autenticado en Keycloak.

---

## **V**

### **Validación de Token**

Proceso mediante el cual Kubernetes verifica que un JWT es válido y fue emitido por un proveedor confiable (Keycloak).

---

## **W**

### **Watch / Get / List**

Permisos de solo lectura en Kubernetes que permiten consultar recursos sin modificarlos.

---

# Resumen

El presente proyecto tuvo como finalidad implementar un esquema de **gestión de identidad centralizada** en Kubernetes mediante la integración de **Keycloak**, **OpenID Connect (OIDC)** y **RBAC**. La solución permitió sustituir un modelo tradicional basado en credenciales locales o certificados manuales por un mecanismo moderno, escalable y alineado con prácticas empresariales de seguridad.

Para lograrlo, se desplegó **Keycloak** dentro del clúster de Kubernetes como proveedor de identidad, configurando un **Realm**, un **Client OIDC**, grupos de usuarios y cuentas de prueba. Posteriormente, el **kube-apiserver** fue configurado para confiar en los tokens emitidos por Keycloak, habilitando autenticación centralizada mediante OIDC.

Sobre esta base, se implementó un esquema de autorización usando **RBAC**, vinculando grupos de Keycloak con **RoleBindings** y **ClusterRoleBindings** en Kubernetes. Esto permitió demostrar distintos niveles de acceso: administración total del clúster, administración limitada a un namespace específico y acceso de solo lectura.

Adicionalmente, se incorporó un stack de observabilidad compuesto por **Jaeger**, **Fluent Bit**, **Loki** y **Grafana**, con el objetivo de fortalecer el análisis de eventos de autenticación, trazabilidad y manejo de logs. Aunque la integración final de logs presentó una limitación asociada a la conectividad y resolución DNS interna del clúster, el despliegue de los componentes fue realizado correctamente y se documentó técnicamente la incidencia.

En conjunto, el proyecto demostró la viabilidad de integrar herramientas de identidad modernas con Kubernetes, mejorando la administración de accesos, la trazabilidad operativa y la seguridad general de la plataforma.

---

# Conclusiones

El desarrollo de este proyecto permitió comprender e implementar un modelo real de **autenticación y autorización centralizada en Kubernetes**, utilizando tecnologías ampliamente adoptadas en entornos empresariales como **Keycloak**, **OIDC** y **RBAC**.

Una de las principales conclusiones es que la integración de identidad con Keycloak representa una mejora significativa frente a mecanismos tradicionales, ya que permite administrar usuarios, grupos y permisos desde un único punto central. Esto simplifica la operación, facilita la escalabilidad del sistema y mejora el control de acceso dentro del clúster.

Asimismo, la configuración de **RBAC** vinculada a grupos de Keycloak demostró ser una estrategia eficiente para separar responsabilidades y aplicar el principio de mínimo privilegio. Gracias a ello, fue posible asignar distintos niveles de acceso según el perfil del usuario, manteniendo una administración ordenada y segura.

Otro aspecto importante fue la validación del flujo de autenticación mediante **kubectl + kubelogin**, lo que permitió comprobar que los usuarios podían autenticarse con sus credenciales de Keycloak y operar dentro de Kubernetes según los permisos otorgados por su grupo. La prueba de cambio dinámico de grupo evidenció además que los permisos pueden modificarse de manera centralizada sin necesidad de recrear usuarios o cambiar configuraciones locales.

En cuanto a observabilidad, el proyecto permitió desplegar y validar herramientas clave como **Jaeger**, **Loki**, **Grafana** y **Fluent Bit**, reforzando la importancia de contar con trazabilidad y centralización de eventos en soluciones modernas. Aunque la visualización final de logs presentó una limitación técnica, el análisis realizado permitió identificar con claridad la causa del problema y documentar adecuadamente el comportamiento del entorno.

Finalmente, este proyecto no solo fortaleció los conocimientos técnicos sobre Kubernetes e identidad digital, sino que también permitió comprender cómo se implementan en la práctica conceptos esenciales de seguridad, gobernanza de accesos y observabilidad en infraestructuras modernas.

---

# Comparativa: OIDC con Keycloak vs certificados X.509 manuales

## Introducción

En Kubernetes existen diferentes mecanismos para autenticar usuarios. Dos de los más representativos son el uso de **OpenID Connect (OIDC) con un proveedor de identidad como Keycloak** y el uso de **certificados X.509 emitidos manualmente**. Ambos enfoques permiten controlar el acceso al clúster, pero presentan diferencias importantes en administración, escalabilidad, seguridad y facilidad operativa.

---

## 1. OIDC con Keycloak

### Descripción

OIDC permite que Kubernetes delegue la autenticación a un proveedor de identidad externo, en este caso **Keycloak**. Los usuarios inician sesión en Keycloak, obtienen un token JWT y Kubernetes valida dicho token para autorizar el acceso según sus claims y grupos.

### Ventajas

- Centraliza la gestión de identidad en una sola plataforma.
- Facilita la administración de usuarios, grupos y credenciales.
- Permite integrar autenticación empresarial moderna.
- Escala mejor cuando existen muchos usuarios.
- Los permisos pueden cambiar dinámicamente mediante grupos.
- Reduce la necesidad de distribuir archivos sensibles manualmente.
- Facilita la auditoría y trazabilidad de autenticación.

### Desventajas

- Requiere mayor configuración inicial.
- Depende de que el proveedor de identidad esté disponible.
- Puede ser más complejo de integrar en laboratorios pequeños.
- Necesita comprender OIDC, tokens y claims para una implementación correcta.

### Casos de uso recomendados

- Empresas o instituciones con múltiples usuarios.
- Ambientes donde se requiere control centralizado de acceso.
- Clústeres con necesidades de crecimiento y delegación administrativa.

- Escenarios donde se busca alineación con estándares modernos de identidad.

---

## 2. Certificados X.509 manuales

### Descripción

Este método consiste en generar manualmente certificados cliente firmados por la autoridad certificadora del clúster. Kubernetes autentica al usuario basándose en la validez del certificado presentado.

### Ventajas

- Es un método nativo de Kubernetes.
- No depende de un proveedor externo de identidad.
- Puede ser útil en ambientes muy pequeños o cerrados.
- Es adecuado para accesos administrativos muy controlados.

### Desventajas

- La administración de certificados es manual y poco flexible.
- Escala mal cuando hay muchos usuarios.
- Revocar o reemplazar certificados puede ser laborioso.
- No ofrece una gestión centralizada de grupos ni credenciales.
- Es menos amigable para usuarios finales.
- Complica auditoría y mantenimiento cuando el equipo crece.

### Casos de uso recomendados

- Entornos pequeños o de laboratorio.
- Accesos administrativos puntuales.
- Infraestructuras cerradas sin necesidad de integración con sistemas de identidad externos.
- Escenarios donde solo unos pocos administradores requieren acceso.

### 3. Diferencias principales

| Criterio                    | OIDC con Keycloak | Certificados X.509 manuales |
|-----------------------------|-------------------|-----------------------------|
| Administración de usuarios  | Centralizada      | Manual                      |
| Escalabilidad               | Alta              | Baja                        |
| Gestión de grupos           | Sí                | Limitada / no nativa        |
| Cambio dinámico de permisos | Sí                | No práctico                 |
| Integración empresarial     | Alta              | Baja                        |
| Complejidad inicial         | Media / Alta      | Media                       |
| Mantenimiento a largo plazo | Más sencillo      | Más costoso                 |
| Experiencia de usuario      | Mejor             | Más rígida                  |

### 4. Análisis comparativo

Desde una perspectiva de seguridad y operación, **OIDC con Keycloak** resulta más adecuado para entornos modernos porque desacopla la identidad del clúster y permite administrar usuarios desde un sistema especializado. Esto facilita la incorporación de nuevos usuarios, la modificación de permisos y la integración con políticas organizacionales.

Por otro lado, los **certificados X.509** siguen siendo un mecanismo válido y robusto, pero su administración manual se vuelve poco práctica cuando aumenta el número de usuarios o cuando se requiere flexibilidad en la autorización.

En este proyecto, la elección de **Keycloak + OIDC** fue adecuada porque permitió demostrar no solo autenticación centralizada, sino también cambios dinámicos de permisos basados en grupos, algo que con certificados manuales sería mucho más complejo de mantener.

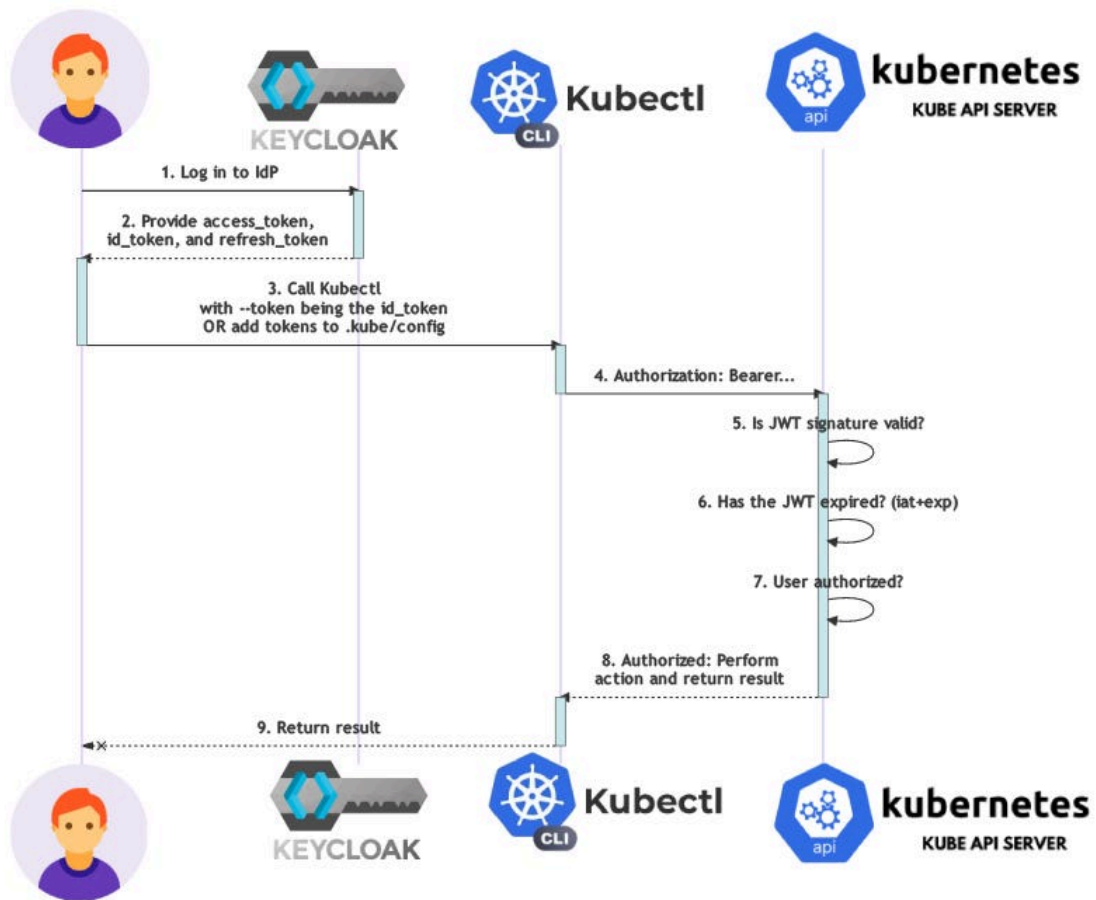
### 5. Conclusión de la comparativa

Aunque ambos métodos permiten autenticar usuarios en Kubernetes, **OIDC con Keycloak** ofrece una solución más moderna, flexible y escalable para entornos reales. En cambio, los **certificados X.509 manuales** son útiles en escenarios limitados, pero presentan mayores retos administrativos y menor adaptabilidad.



Por ello, para organizaciones que buscan seguridad, gobernanza y crecimiento ordenado, la mejor alternativa suele ser integrar Kubernetes con un proveedor de identidad mediante **OIDC**.

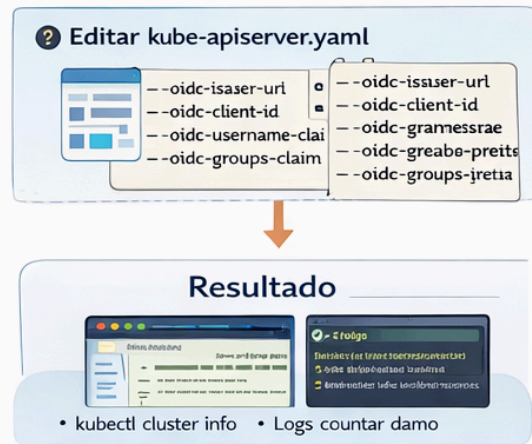




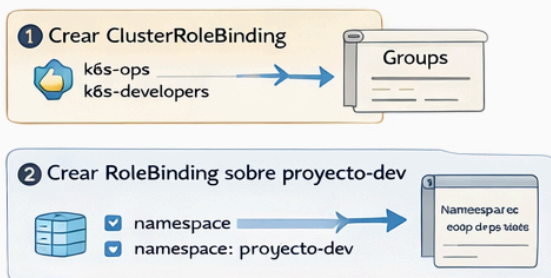
### Módulo 1 Desplegar Keycloak en K8s



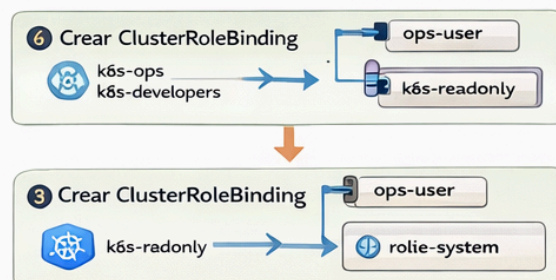
### Módulo 2 Configuración de Keycloak



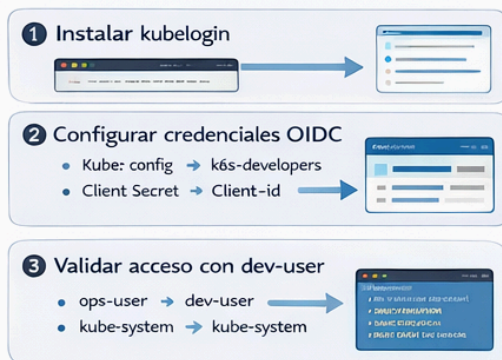
### Módulo 3 RBAC mapeado a grupos de Keycloak



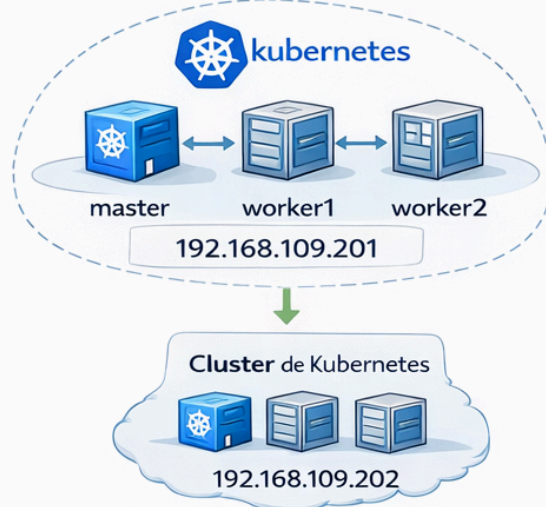
### Módulo 4 RBAC mapeado a grupos de Keycloak

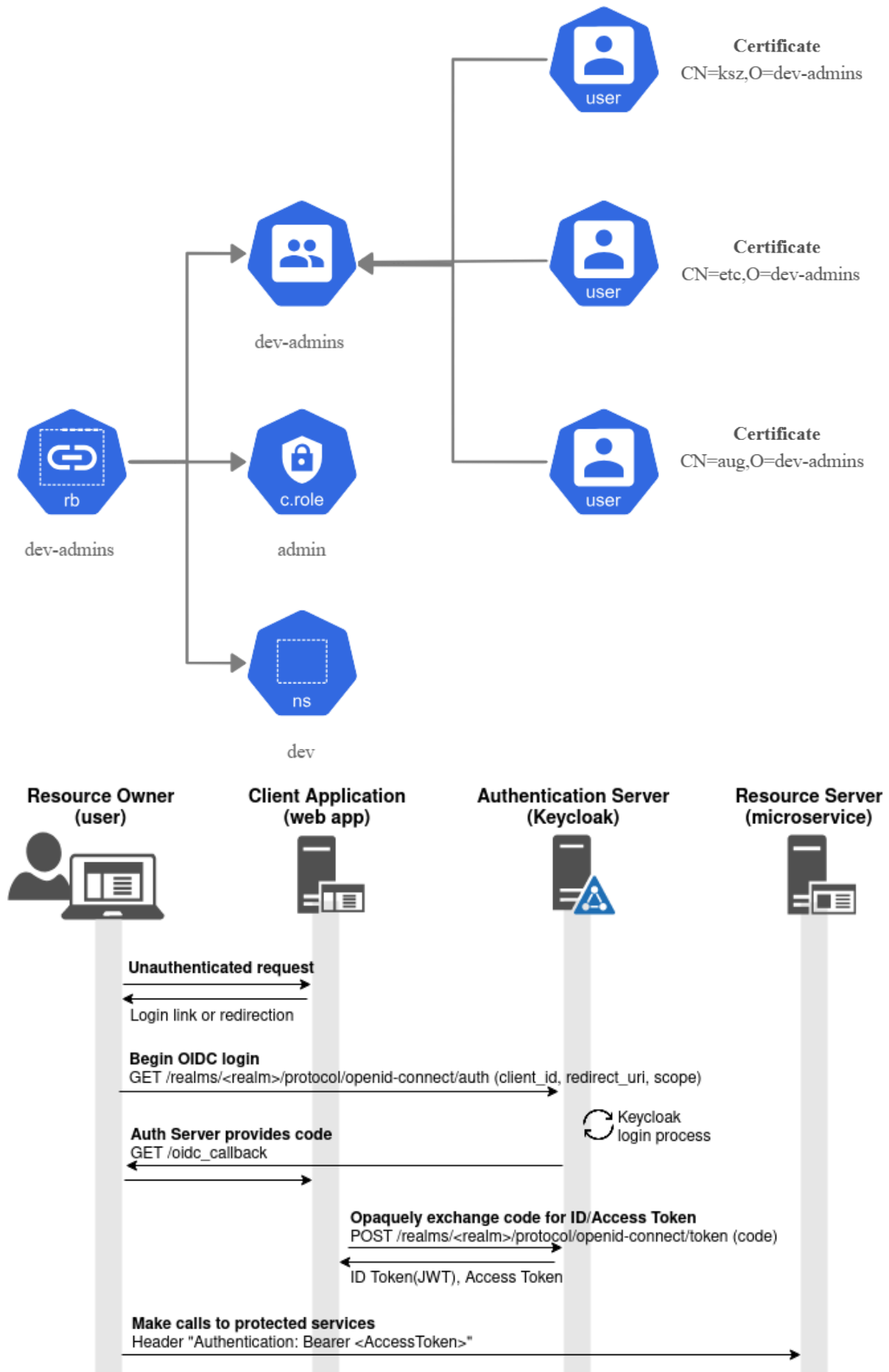


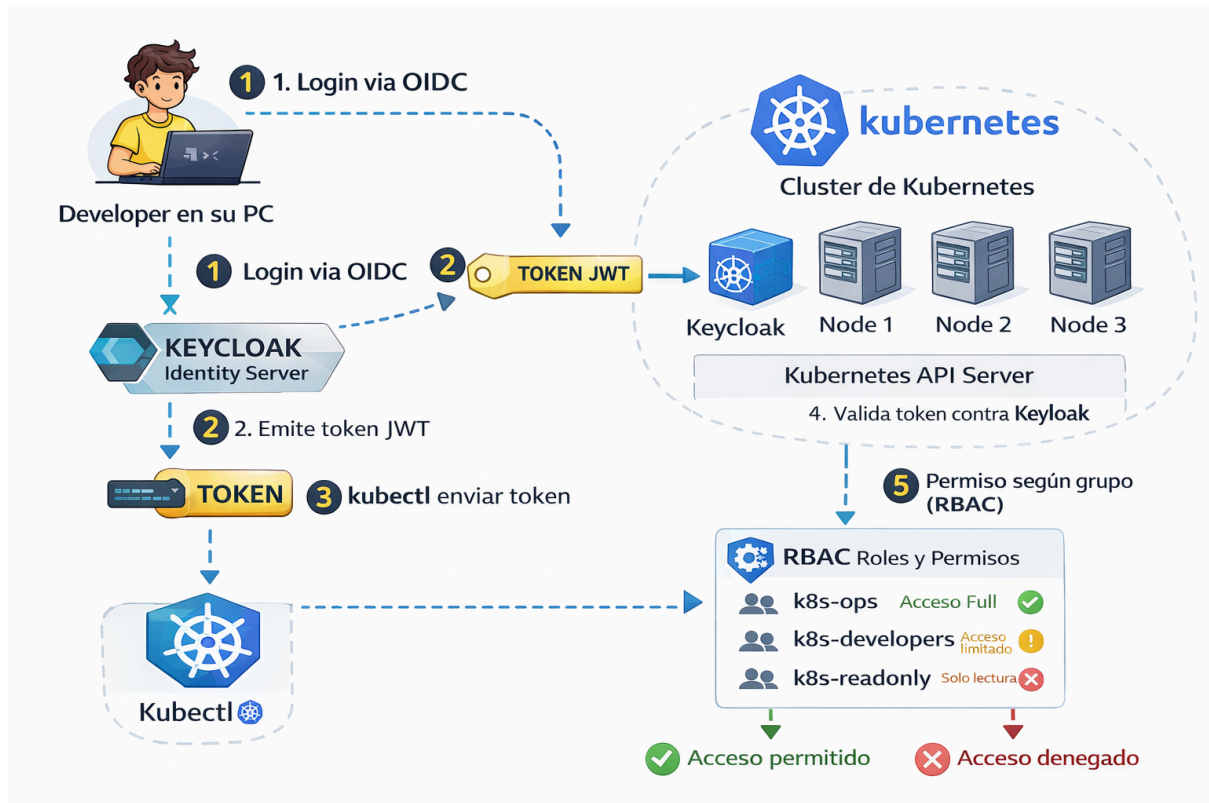
### Módulo 5 Autenticación con kubectl vía OIDC



### Módulo 7 Cluster de Kubernetes







## Arquitectura objetivo – Keycloak + OIDC + Kubernetes RBAC

Equipo 3 · Proyecto Final · Seguridad en Infraestructura y Kubernetes

